



DESARROLLO DE APLICACIONES WEB

PROTOTIPO DE PROYECTO

PÁGINA WEB DE PETCARE

Autores

Marina Collazo Couñago

Guillermo Fernández Castro

Andrés Daparte Cabo

Febrero 2026

Índice

1. Introducción	2
2. Inventario de contenido	2
3. Storyboards	4
4. Arquitectura de información	5
5. Mapa de navegación	7
6. Casos de uso abordados	8
7. Guía de estilos y aspectos técnicos	9
7.1. Paleta de colores	9
7.1.1. Colores base	9
7.1.2. Colores de llamada a la acción	9
7.1.3. Colores para tipografías y tonos neutros	9
7.2. Tipografías	9
7.3. Identidad visual en el diseño	10
8. Interfaces	11
8.1. Bocetos a mano alzada	11
8.2. Wireframes	16
8.2.1. Página principal	16
8.2.2. Clínicas	17
8.2.3. Urgencias	18
8.2.4. Servicios y tratamientos	19
8.2.5. Servicio específico	20
8.2.6. Sobre nosotros	21
8.2.7. Consejos de salud	22
8.2.8. Consejo específico	23
8.2.9. Solicitud de cita	24
8.3. Mockups	25
8.3.1. Página principal	25
8.3.2. Clínicas	26
8.3.3. Urgencias	27
8.3.4. Servicios y tratamientos	28
8.3.5. Consejos de salud	29
8.3.6. Sobre nosotros	30
8.3.7. Servicio específico	31
8.3.8. Solicitud de cita	32
9. Estructura de ficheros	34
10. Mapas de etiquetas HTML	35
10.1. Página principal	35
10.2. Clínicas	37

10.3. Urgencias	39
10.4. Servicios y tratamientos	41
10.5. Servicio específico	43
10.6. Sobre nosotros	45
10.7. Consejos de salud	47
10.8. Consejo específico	49
10.9. Solicitud de cita	51
11.Práctica 3: Responsividad	52
11.1. Flexible Grids con <code>float</code> — Clínicas	53
11.2. Diseño multicolumna — Sobre nosotros	56
11.3. CSS Flex Container — Solicitud de cita	59
11.4. CSS Grid — Servicios	62
11.5. Framework Bootstrap — Consejos de salud	65
12.Práctica 4: JavaScript	68
12.1. Efectos visibles	68
12.1.1. Navegación dinámica por pestañas en la página Sobre Nosotros	68
12.1.2. Búsqueda dinámica de clínicas	71
12.1.3. Rotación dinámica de consejos aleatorios en la página de inicio	74
12.1.4. Autocompletado dinámico del nombre de la clínica en pedir cita	77
12.2. Métodos de acceso al DOM	79
12.3. Eventos implementados	79
12.4. Sintaxis ECMAScript 6	79
12.5. Uso de objetos jQuery	80
12.6. Carga de contenido en formato XML	82
12.6.1. Método utilizado: Objeto <code>XMLHttpRequest</code>	82
12.7. Carga de contenido en formato JSON	84
12.7.1. Método utilizado: objeto jQuery (<code>\$.getJSON</code>)	84

1. Introducción

Tema propuesto: Página web para la franquicia de clínicas veterinarias “PetCare”.

Objetivo del proyecto: Digitalizar la presencia de dicha franquicia e informar y facilitar sus servicios para alcanzar una mayor clientela, ampliando el alcance de la empresa.

Público: Dueños de mascotas que requieran de cuidados, atención de urgencia, mantenimiento u otros servicios relacionados al bienestar y salud de la mascota.

Alcance: El proyecto tiene como objetivo el desarrollo de un frontend mediante maquetación básica de interfaces web, implementando una navegación funcional entre por lo menos 5 páginas alternativas.

El proyecto abarca el diseño y desarrollo del frontend de un sitio web fundamentalmente informativo para una franquicia de clínicas veterinarias. El entregable final consistirá en un prototipo funcional de al menos 9 páginas enlazadas entre sí, involucrando la maquetación visual de las interfaces de usuario de dichas páginas.

2. Inventario de contenido

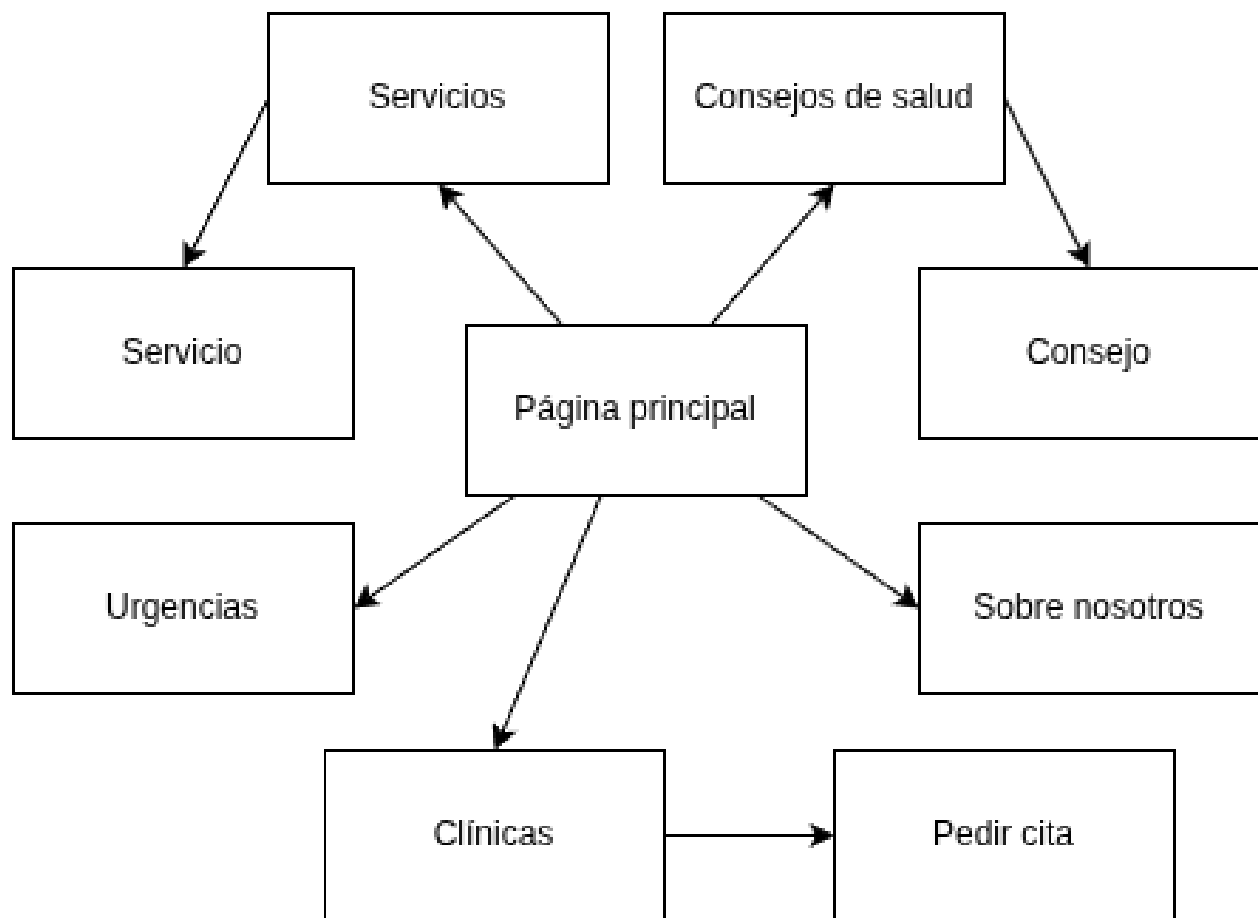


Figura 1: Inventario de contenido

Páginas	Elementos clave a exponer	Objetivo del usuario
Página Principal	■ Buscador global de clínicas. ■ Botón destacado de Urgencias. ■ Accesos rápidos a Servicios y Consejos.	Acceder rápidamente a la información crítica o iniciar la navegación.
Urgencias	■ Listado filtrado: solo clínicas abiertas ahora. ■ Botones de llamada directa. ■ Aviso de “Llamar antes de acudir”.	Obtener atención veterinaria inmediata sin fricción.
Clínicas	■ Buscador por ubicación. ■ Fichas con dirección y horario. ■ Formulario emergente para reservar.	Localizar un centro y realizar la transacción (reservar cita).
Servicios y servicio específico	■ Catálogo de tratamientos (Cirugía, Rayos X...). ■ Páginas de detalle con explicación del procedimiento.	Informarse sobre qué tratamientos ofrece la franquicia.
Consejos de Salud y consejo específico	■ Artículos educativos. ■ Filtros por especie (Perros/Gatos). ■ Enlaces cruzados para pedir cita.	Resolver dudas sobre cuidados y prevención (fidelización).
Sobre Nosotros	■ Filosofía y valores (Sostenibilidad). ■ Información corporativa.	Generar confianza y credibilidad en la marca.

Cuadro 1: Inventario de contenidos, formato tabla

3. Storyboards

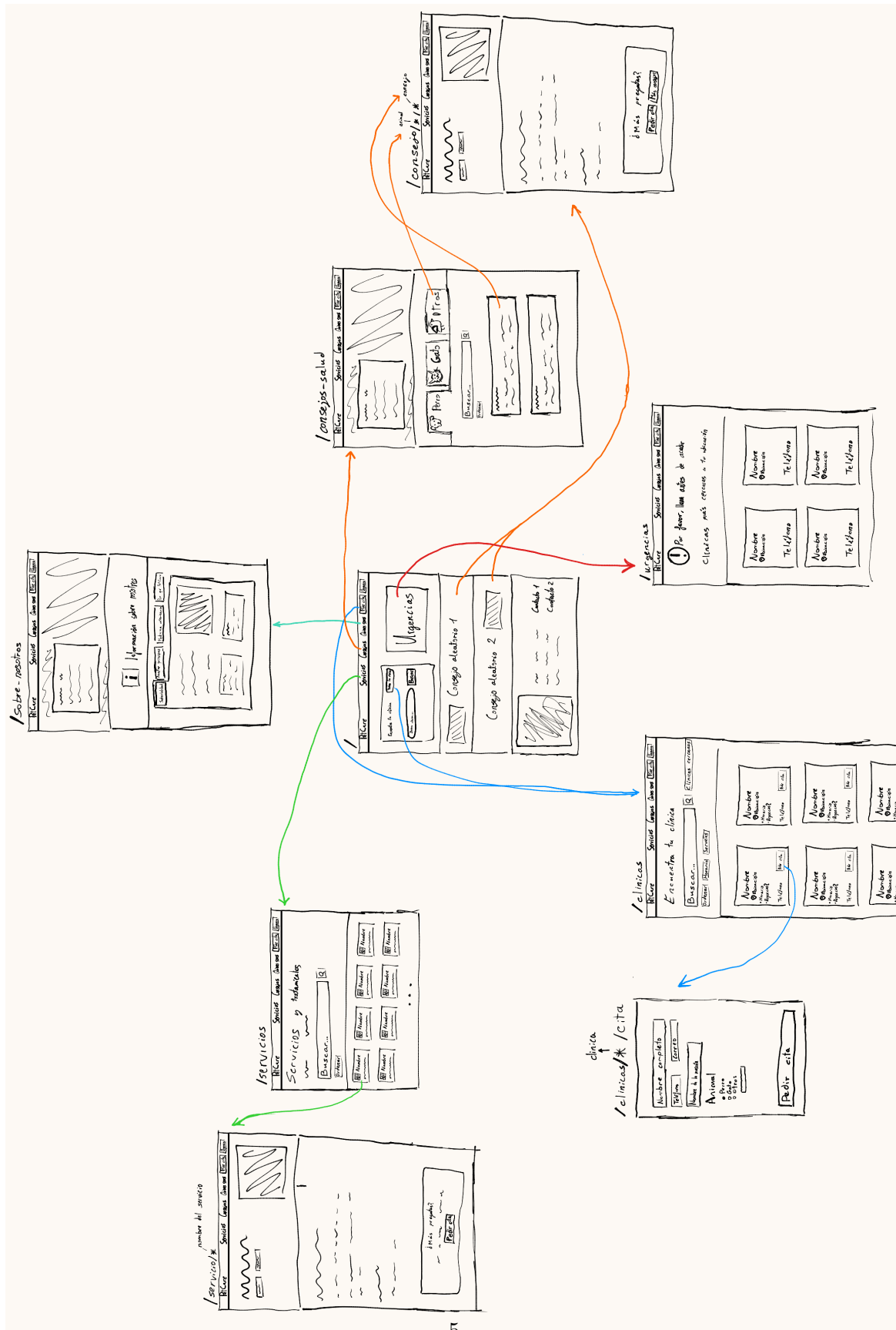


Figura 2: Storyboard

4. Arquitectura de información

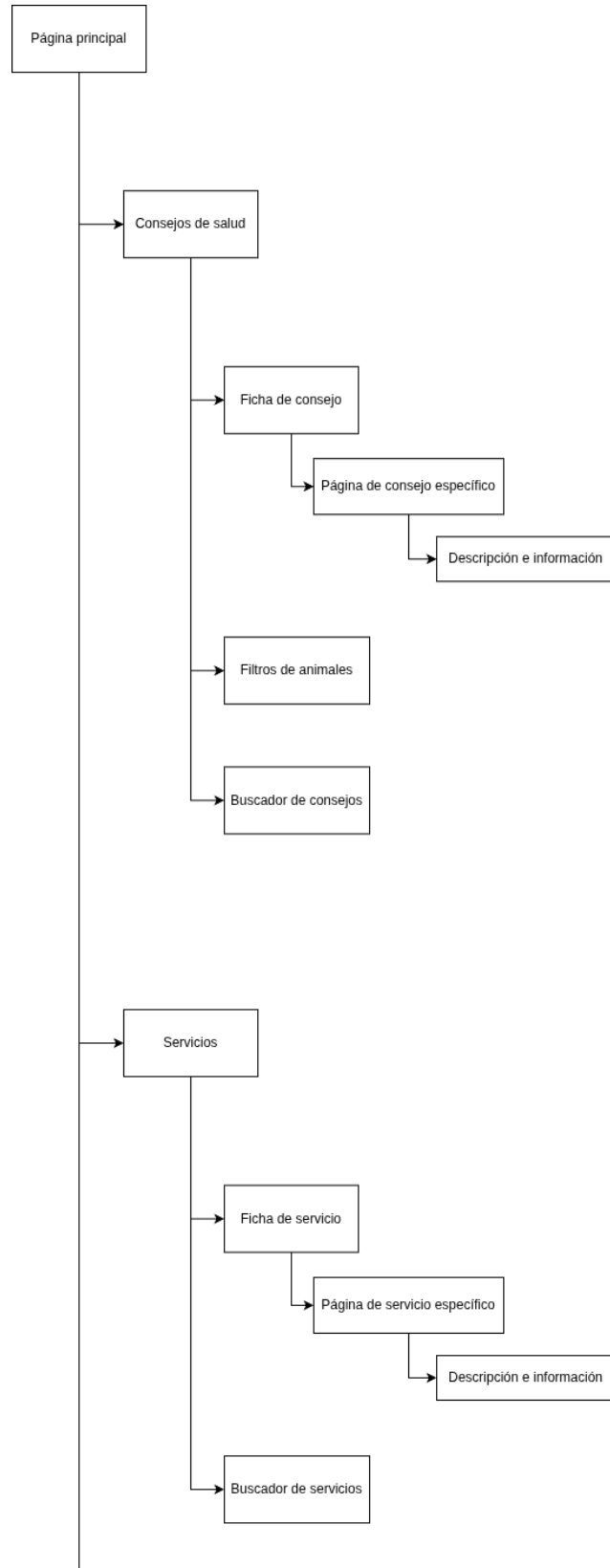


Figura 3: Arquitectura de información (1/3)

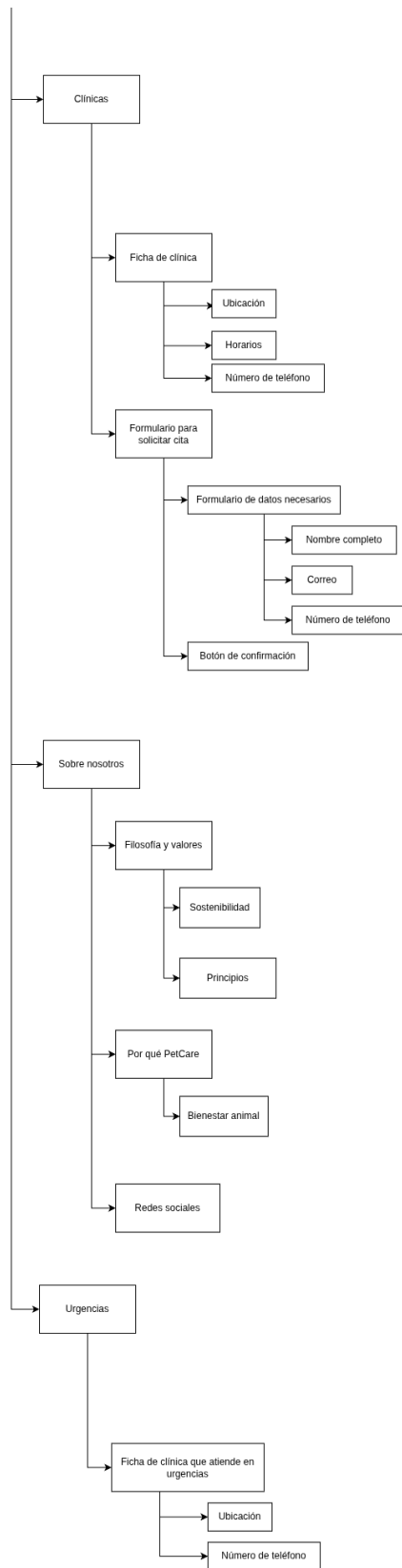


Figura 4: Arquitectura de información (2/2)

5. Mapa de navegación

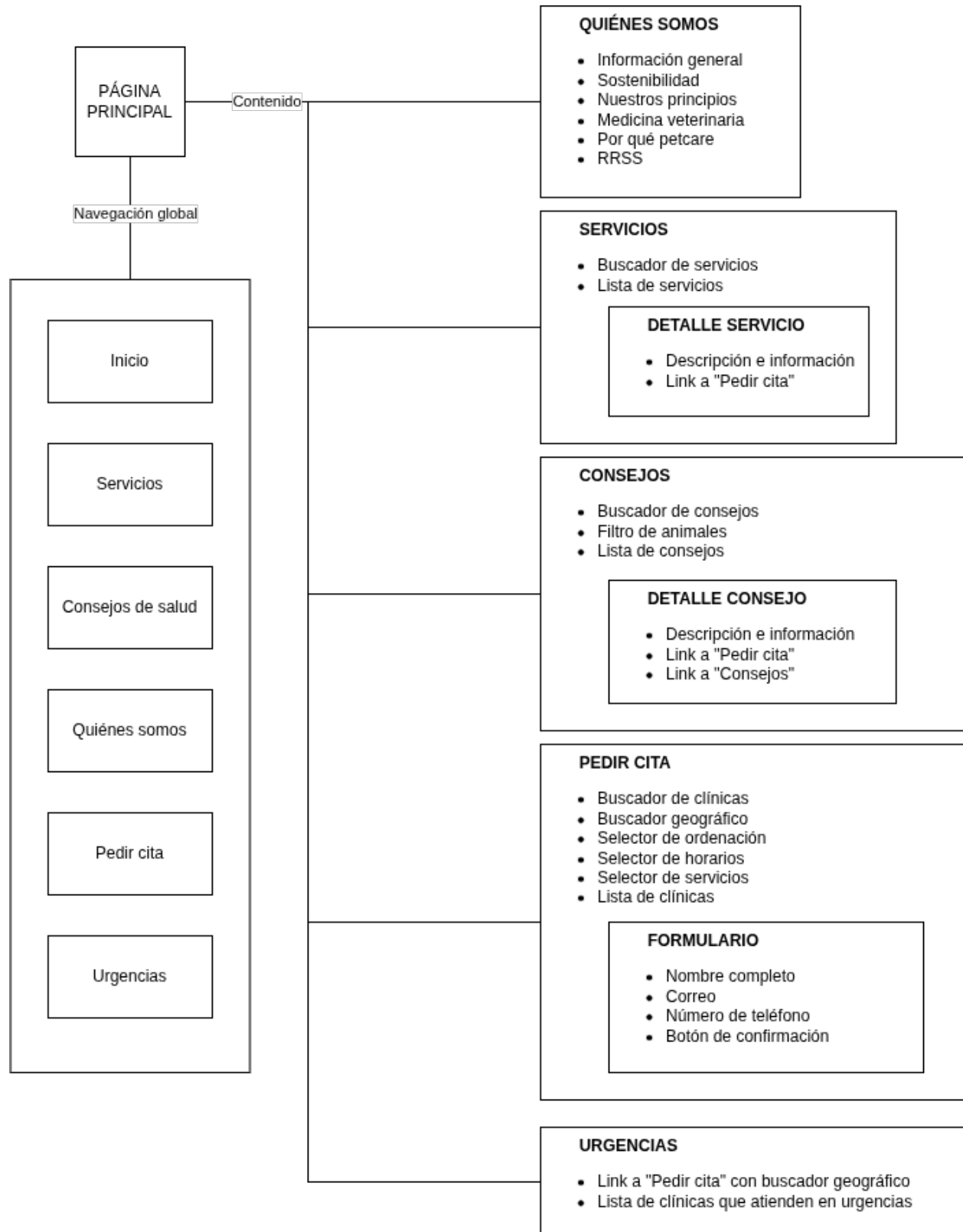


Figura 5: Mapa de navegación

6. Casos de uso abordados

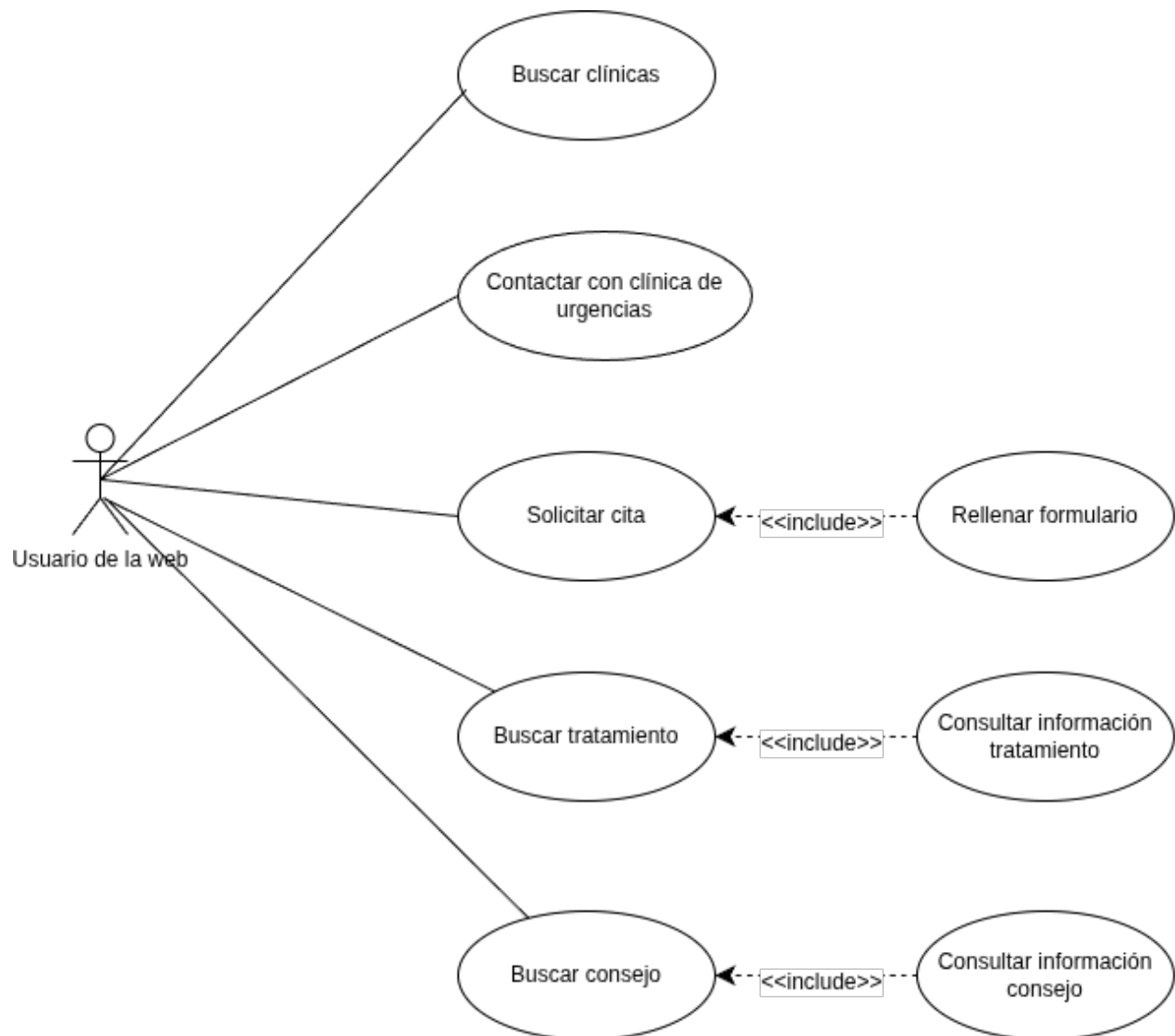


Figura 6: Casos de uso

7. Guía de estilos y aspectos técnicos

El diseño de visual de PetCare está fundamentado en proporcionar una estilización de la identidad y asociarse a la psicología del color sanitaria priorizando unos tonos más pasteles, reduciendo la posibilidad de fatiga visual y transmitiendo calma. Se reservan colores más saturados únicamente para las llamadas a acción para guiar la atención del usuario, como sucede por ejemplo en la página principal con el botón grande de “Urgencias”.

7.1. Paleta de colores

Se definió un esquema cromático dividido en tres categorías funcionales:

7.1.1. Colores base

- #ADB2D4: Usado como acento primario, para estados “seleccionado” y filtros activos.
- #C7D9DD: Utilizado en fondos con degradado suave.
- #D5E5D5: Aplicado en fondos de sección alternativos y badges (etiquetas) de categoría.
- #EEF1DA: Fondos cálidos para secciones informativas ligeras.

7.1.2. Colores de llamada a la acción

- #EF5A6F: Color Crítico. Reservado exclusivamente para el botón flotante y avisos de “Urgencias”.
- #7B82A8: Botones de acción principal (CTA), como “Pedir cita” o “Buscar”.
- #88C9A1: Acciones secundarias o de confirmación (ej: “Ver clínicas cercanas”).

7.1.3. Colores para tipografías y tonos neutros

- Texto Principal (#2A2D3F): Tono Dark Charcoal para títulos y lectura principal.
- Texto Secundario (#4A4a5E): Tono Medium Gray para cuerpo de texto, descripciones y etiquetas.
- Fondos Neutros: Blanco puro (#FFFFFF) para tarjetas y contenedores; Off-white (#F9FAFB) para fondos de inputs en formularios.

7.2. Tipografías

Se utilizan principalmente dos familias tipográficas de Google Fonts para distinguir claramente la estructura de la página.

Para Títulos: Poppins (Sans-serif geométrica)

Peso: Utilizada exclusivamente en seminegrita.

Propósito: Aportar modernidad y estilizar.

Aplicación: Encabezados h1, h2, h3 y títulos de tarjetas.

Para Cuerpo: Inter (Sans-serif humanista)

Pesos: Utilizada en diversos pesos.

Propósito: Aportar mayor legibilidad independientemente del tamaño de la pantalla del dispositivo.

Aplicación: Párrafos, descripciones, etiquetas de formularios y textos de botones.

7.3. Identidad visual en el diseño

Como rasgo relativamente distintivo de la interfaz de usuario se han definido múltiples bordes asimétricos suavizando la rigidez de una maquetación con bordes más afilados.

8. Interfaces

8.1. Bocetos a mano alzada

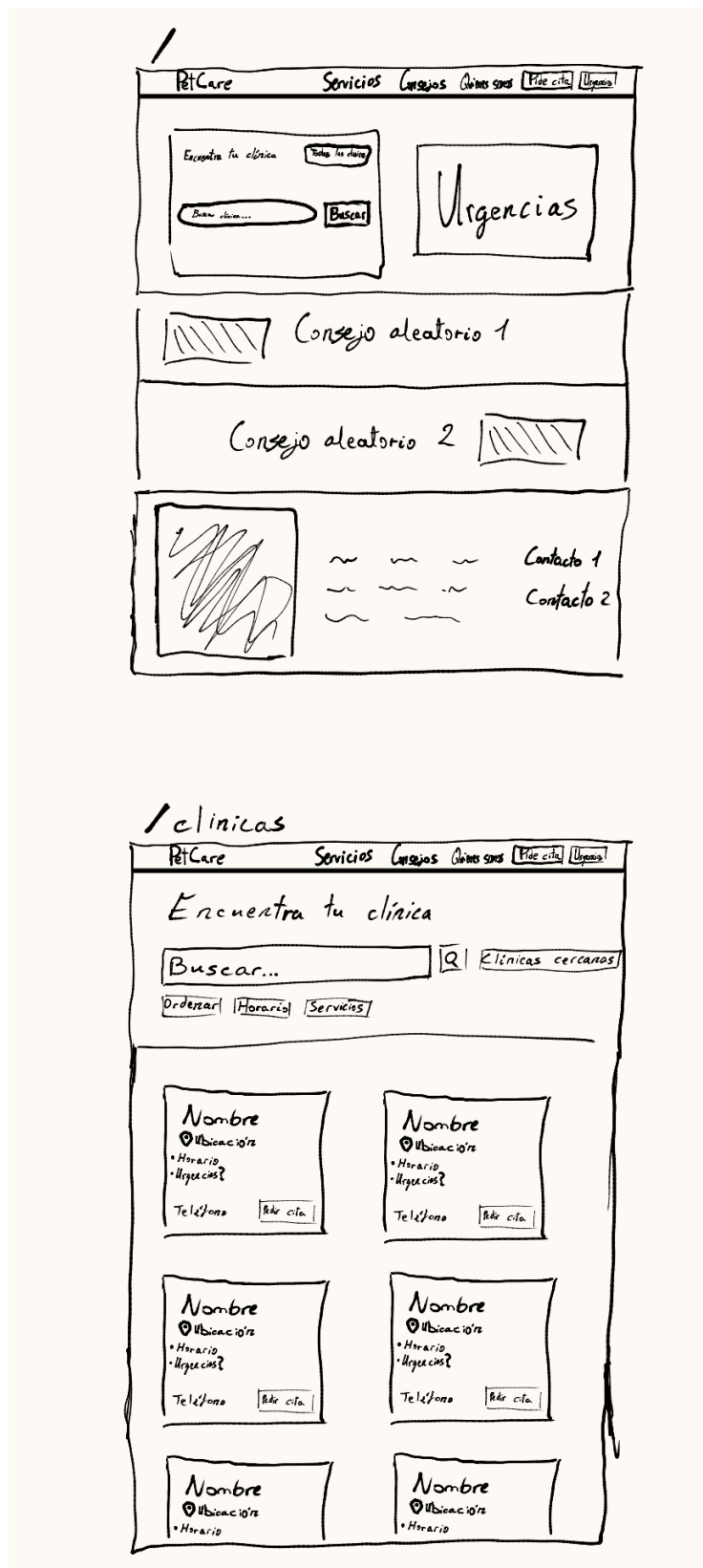


Figura 7: Bocetos (1/5)

clínica
↑
/clínicas/* /cita

Nombre completo

Teléfono Correo

Nombre de la mascota

Animal

- ☐ Perro
- ☐ Gato
- ☐ Otro:

Pedir cita

Urgencias

PetCare
Servicios
Consejos
Quitar spots
Pide cita
Urgencias

❗ Por favor, llama antes de acudir

Clinicas más cercanas a tu ubicación

Nombre Ubicación	Nombre Ubicación
Teléfono	Teléfono
Nombre Ubicación	Nombre Ubicación
Teléfono	Teléfono

Figura 8: Bocetos (2/5)

/servicios

PetCare Servicios Consejos Quieres saber Pide cita Urgencia

Servicios y tratamientos

Buscar...

Ordenar

Nombre	Nombre	Nombre	Nombre
Nombre	Nombre	Nombre	Nombre

...

/consejos-salud

PetCare Servicios Consejos Quieres saber Pide cita Urgencia

Buscar...

Ordenar

Figura 9: Bocetos (3/5)

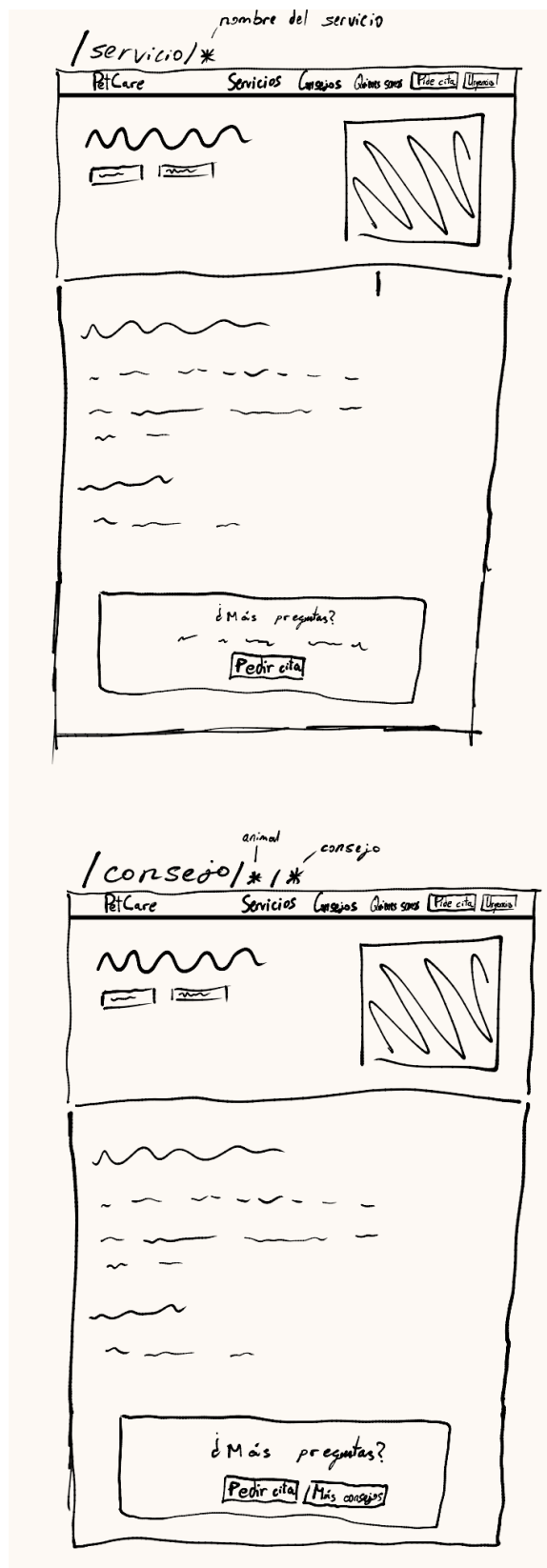


Figura 10: Bocetos (4/5)



Figura 11: Bocetos (5/5)

8.2. Wireframes

8.2.1. Página principal

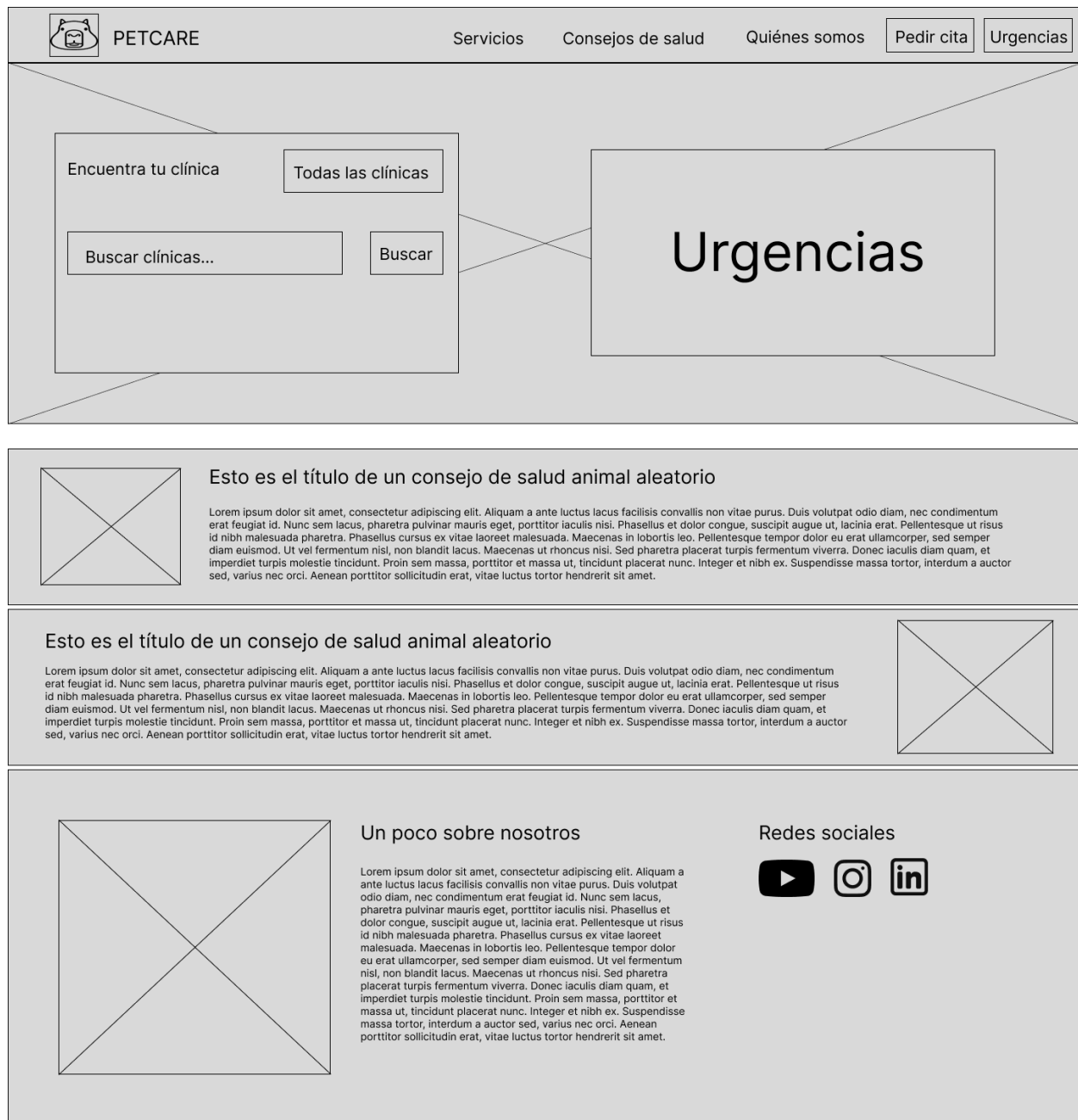


Figura 12: Wireframe de página principal

8.2.2. Clínicas

 PETCARE

ServiciosConsejos de saludQuiénes somosPedir citaUrgencias

Encuentra tu clínica

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Aliquam a ante luctus lacus facilisis convallis non vitae purus. Duis volutpat odio diam, nec condimentum erat feugiat id.

Buscar servicios...

Buscar

Clínicas más cercanas

Ordenar

Horarios

Servicios



Nombre
Ubicación

Horario de atención al cliente

09:00 - 20:30

 +34 999 99 99 99

Pedir cita



Nombre
Ubicación

Esta clínica atiende urgencias

Horario de atención al cliente

09:00 - 20:30

 +34 999 99 99 99

Pedir cita



Nombre
Ubicación

Horario de atención al cliente

09:00 - 20:30

 +34 999 99 99 99

Pedir cita



Nombre
Ubicación

Horario de atención al cliente

09:00 - 20:30

 +34 999 99 99 99

Pedir cita

Figura 13: Wireframe de página de clínicas

8.2.3. Urgencias



Figura 14: Wireframe de página de urgencias

8.2.4. Servicios y tratamientos

 PETCARE

ServiciosConsejos de saludQuiénes somosPedir citaUrgencias

Servicios y tratamientos

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Aliquam a ante luctus lacus facilisis convallis non vitae purus. Duis volutpat odio diam, nec condimentum erat feugiat id.

Buscar servicios...

Buscar

Ordenar



Nombre

Lorem ipsum dolor sit amet, consectetur adipiscing elit.

Aliquam a ante luctus lacus facilisis convallis non vitae purus. Duis volutpat odio diam, nec condimentum erat feugiat id.



Nombre

Lorem ipsum dolor sit amet, consectetur adipiscing elit.

Aliquam a ante luctus lacus facilisis convallis non vitae purus. Duis volutpat odio diam, nec condimentum erat feugiat id.



Nombre

Lorem ipsum dolor sit amet, consectetur adipiscing elit.

Aliquam a ante luctus lacus facilisis convallis non vitae purus. Duis volutpat odio diam, nec condimentum erat feugiat id.



Nombre

Lorem ipsum dolor sit amet, consectetur adipiscing elit.

Aliquam a ante luctus lacus facilisis convallis non vitae purus. Duis volutpat odio diam, nec condimentum erat feugiat id.



Nombre

Lorem ipsum dolor sit amet, consectetur adipiscing elit.



Nombre

Lorem ipsum dolor sit amet, consectetur adipiscing elit.

Figura 15: Wireframe de página de servicios y tratamientos

8.2.5. Servicio específico

 PETCARE

Servicios

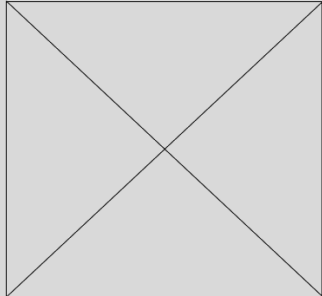
Consejos de salud

Quiénes somos

Pedir cita

Urgencias

Esto es el nombre de un servicio aleatorio



1. Lorem Ipsum

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Aliquam a ante luctus lacus facilisis convallis non vitae purus. Duis volutpat odio diam, nec condimentum erat feugiat id. Nunc sem lacus, pharetra pulvinar mauris eget, porttitor iaculis nisi. Phasellus et dolor congue, suscipit augue ut, lacinia erat. Pellentesque ut risus id nibh malesuada pharetra. Phasellus cursus ex vitae laoreet malesuada. Maecenas in lobortis leo. Pellentesque tempor dolor eu erat ullamcorper, sed semper diam euismod. Ut vel fermentum nisl, non blandit lacus. Maecenas ut rhoncus nisi. Sed pharetra placerat turpis fermentum viverra. Donec iaculis diam quam, et imperdiet turpis molestie tincidunt. Proin sem massa, porttitor et massa ut, tincidunt placerat nunc. Integer et nibh ex. Suspendisse massa tortor, interdum a auctor sed, varius nec orci. Aenean porttitor sollicitudin erat, vitae luctus tortor hendrerit sit amet.

2. Lorem Ipsum

Vivamus mauris est, ullamcorper eget rutrum eu, blandit ac justo. Fusce convallis consectetur quam. Interdum et malesuada fames ac ante ipsum primis in faucibus. Nunc tincidunt neque eleifend varius euismod. Aenean ac metus in turpis sodales faucibus nec eget diam. Phasellus non ante nec metus euismod pretium vel eu tellus. Integer aliquam mi nec ullamcorper tincidunt. Vestibulum sed auctor dui. Nulla facilisi. Pellentesque fringilla, orci a scelerisque bibendum, ex dolor luctus arcu, vel tincidunt dui nulla sit amet lorem. Ut nec ex hendrerit, faucibus ipsum ut, vehicula risus. Aliquam interdum nisl ipsum, ac suscipit metus iaculis placerat. Etiam commodo in mauris in semper. Sed in tempor dolor. Praesent a vulputate velit. Vestibulum non rutrum arcu. Donec hendrerit eleifend dolor vitae volutpat. Aliquam viverra elit neque, non vulputate augue convallis ut. Nullam iaculis ornare sem, ac pretium ex consequat ac. Aliquam mattis lacus eu semper aliquam.

3. Lorem Ipsum

Pellentesque id ante eget felis accumsan tincidunt. Donec imperdiet non mi gravida commodo. Vivamus imperdiet laoreet vulputate. Sed interdum nisi eu dolor tincidunt, quis pharetra lectus dictum. Nullam iaculis ullamcorper justo, vel rutrum lacus tempor finibus. In eleifend fermentum venenatis. Integer luctus iaculis quam a maximus. Ut sagittis leo felis, in rutrum arcu blandit sit amet.

¿Tienes más preguntas?

Pide cita en tu clínica más cercana para encontrar las respuestas que buscas

Pedir cita

Figura 16: Wireframede la página de un servicio específico

8.2.6. Sobre nosotros

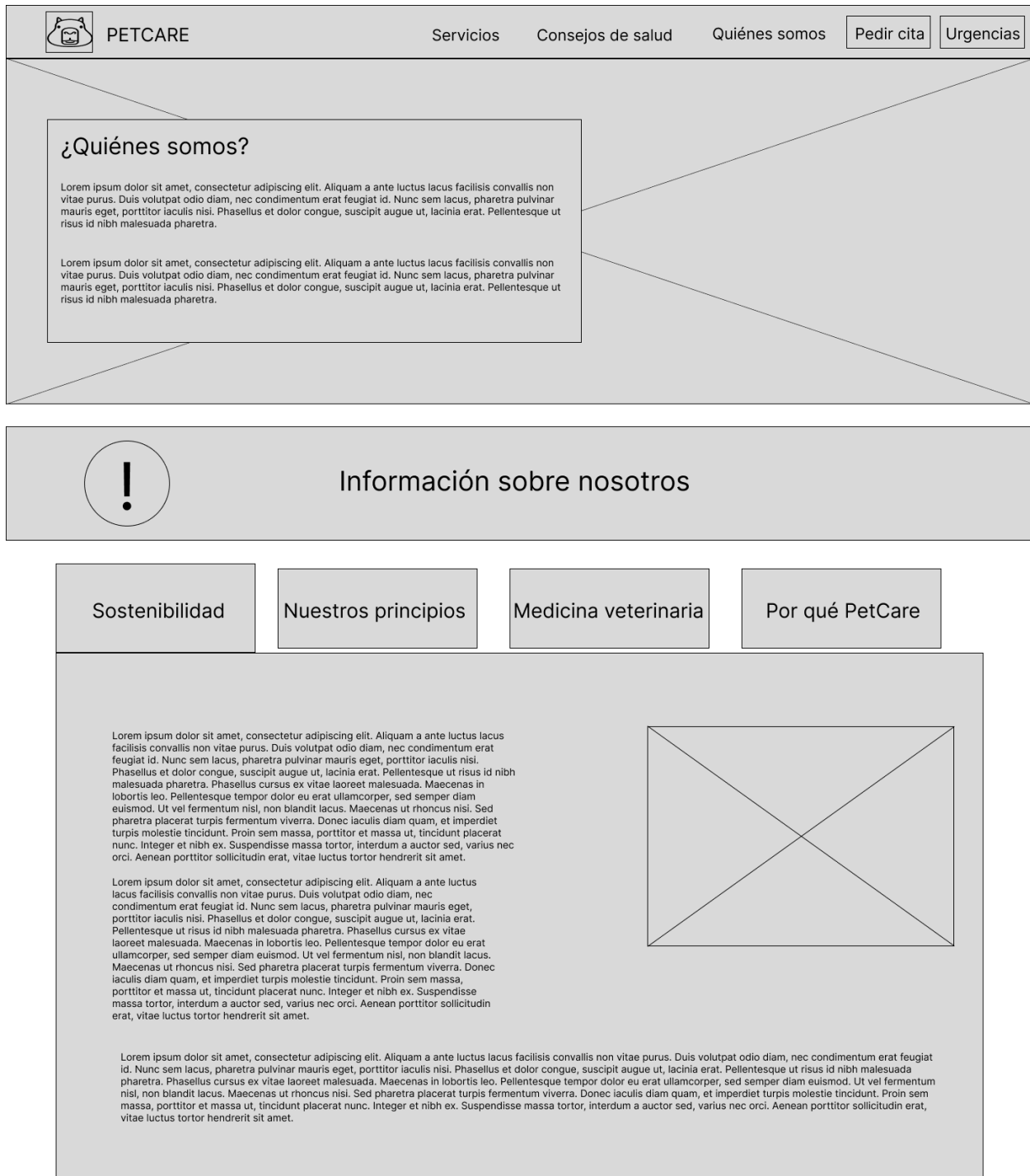


Figura 17: Wireframe de página de información sobre la franquicia

8.2.7. Consejos de salud

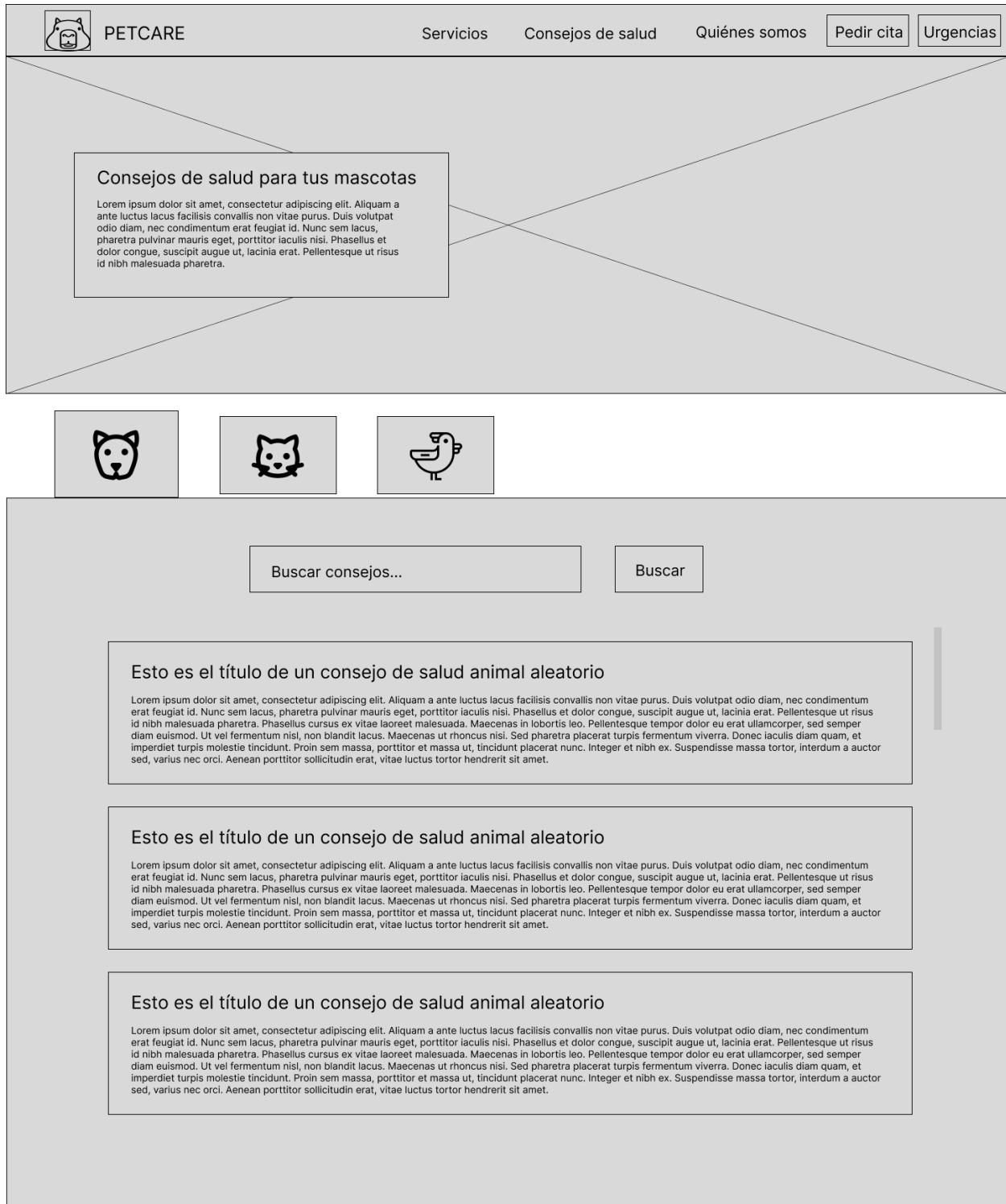


Figura 18: Wireframe de la página de consejos de salud para mascotas

8.2.8. Consejo específico

 PETCARE

Servicios

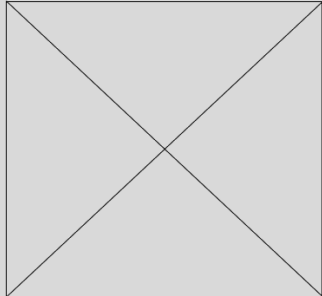
Consejos de salud

Quiénes somos

Pedir cita

Urgencias

Esto es el título de un consejo de salud animal aleatorio



1. Lorem Ipsum

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Aliquam a ante luctus lacus facilisis convallis non vitae purus. Duis volutpat odio diam, nec condimentum erat feugiat id. Nunc sem lacus, pharetra pulvinar mauris eget, porttitor iaculis nisi. Phasellus et dolor congue, suscipit augue ut, lacinia erat. Pellentesque ut risus id nibh malesuada pharetra. Phasellus cursus ex vitae laoreet malesuada. Maecenas in lobortis leo. Pellentesque tempor dolor eu erat ullamcorper, sed semper diam euismod. Ut vel fermentum nisl, non blandit lacus. Maecenas ut rhoncus nisi. Sed pharetra placerat turpis fermentum viverra. Donec iaculis diam quam, et imperdiet turpis molestie tincidunt. Proin sem massa, porttitor et massa ut, tincidunt placerat nunc. Integer et nibh ex. Suspendisse massa tortor, interdum a auctor sed, varius nec orci. Aenean porttitor sollicitudin erat, vitae luctus tortor hendrerit sit amet.

2. Lorem Ipsum

Vivamus mauris est, ullamcorper eget rutrum eu, blandit ac justo. Fusce convallis consectetur quam. Interdum et malesuada fames ac ante ipsum primis in faucibus. Nunc tincidunt neque eleifend varius euismod. Aenean ac metus in turpis sodales faucibus nec eget diam. Phasellus non ante nec metus euismod pretium vel eu tellus. Integer aliquam mi nec ullamcorper tincidunt. Vestibulum sed auctor dui. Nulla facilisi. Pellentesque fringilla, orci a scelerisque bibendum, ex dolor luctus arcu, vel tincidunt dui nulla sit amet lorem. Ut nec ex hendrerit, faucibus ipsum ut, vehicula risus. Aliquam interdum nisl ipsum, ac suscipit metus iaculis placerat. Etiam commodo in mauris in semper. Sed in tempor dolor. Praesent a vulputate velit. Vestibulum non rutrum arcu. Donec hendrerit eleifend dolor vitae volutpat. Aliquam viverra elit neque, non vulputate augue convallis ut. Nullam iaculis ornare sem, ac pretium ex consequat ac. Aliquam mattis lacus eu semper aliquam.

3. Lorem Ipsum

Pellentesque id ante eget felis accumsan tincidunt. Donec imperdiet non mi gravida commodo. Vivamus imperdiet laoreet vulputate. Sed interdum nisi eu dolor tincidunt, quis pharetra lectus dictum. Nullam iaculis ullamcorper justo, vel rutrum lacus tempor finibus. In eleifend fermentum venenatis. Integer luctus iaculis quam a maximus. Ut sagittis leo felis, in rutrum arcu blandit sit amet.

¿Tienes más preguntas?

Pedir cita

Más consejos

Figura 19: Wireframe de la página de un consejo de salud específico

8.2.9. Solicitud de cita

Nombre de la clínica

 Ubicación

Nombre completo

Teléfono

Correo electrónico

Nombre de la mascota







Pedir cita

Figura 20: Wireframe del formulario para “Pedir cita”

8.3. Mockups

8.3.1. Página principal

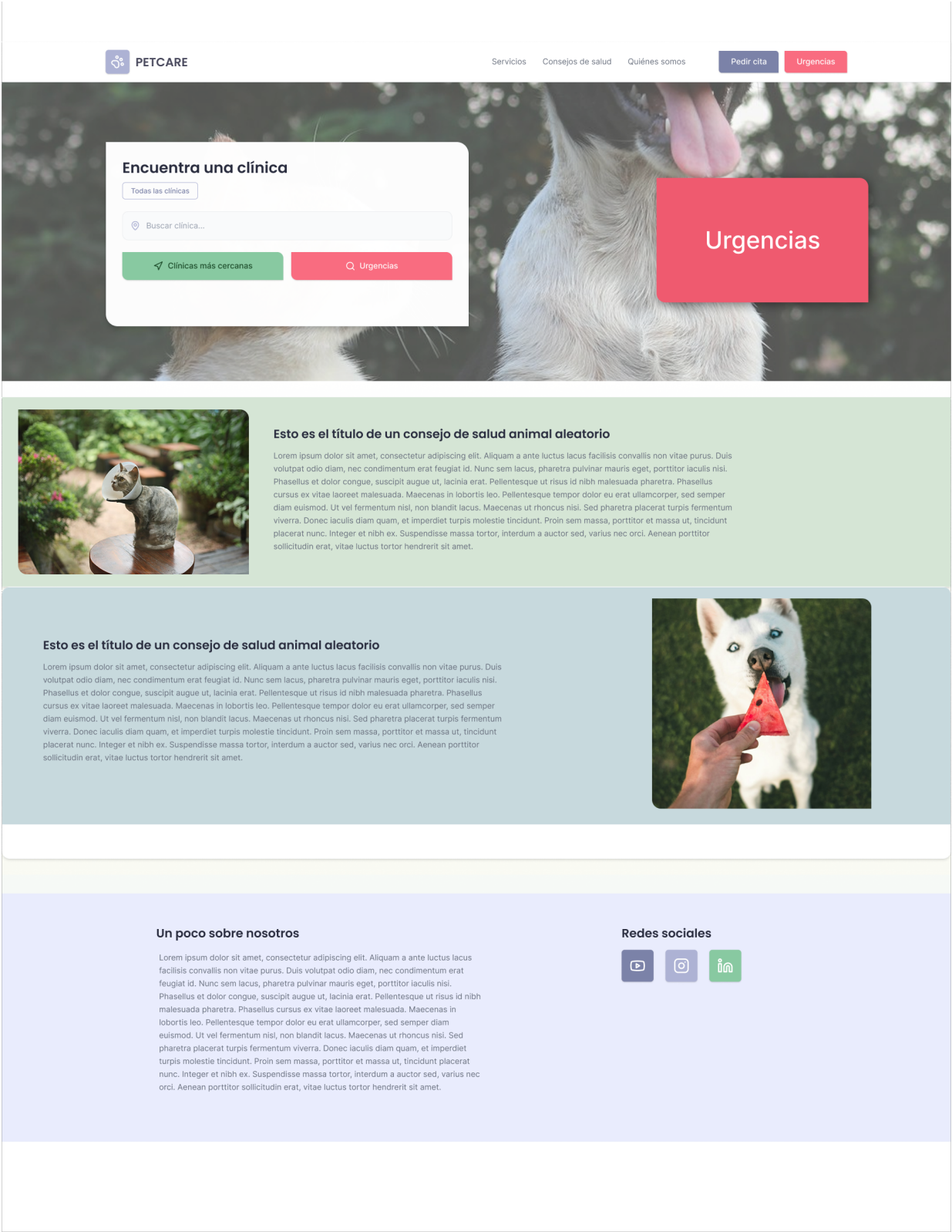


Figura 21: Mockup de página principal

8.3.2. Clínicas

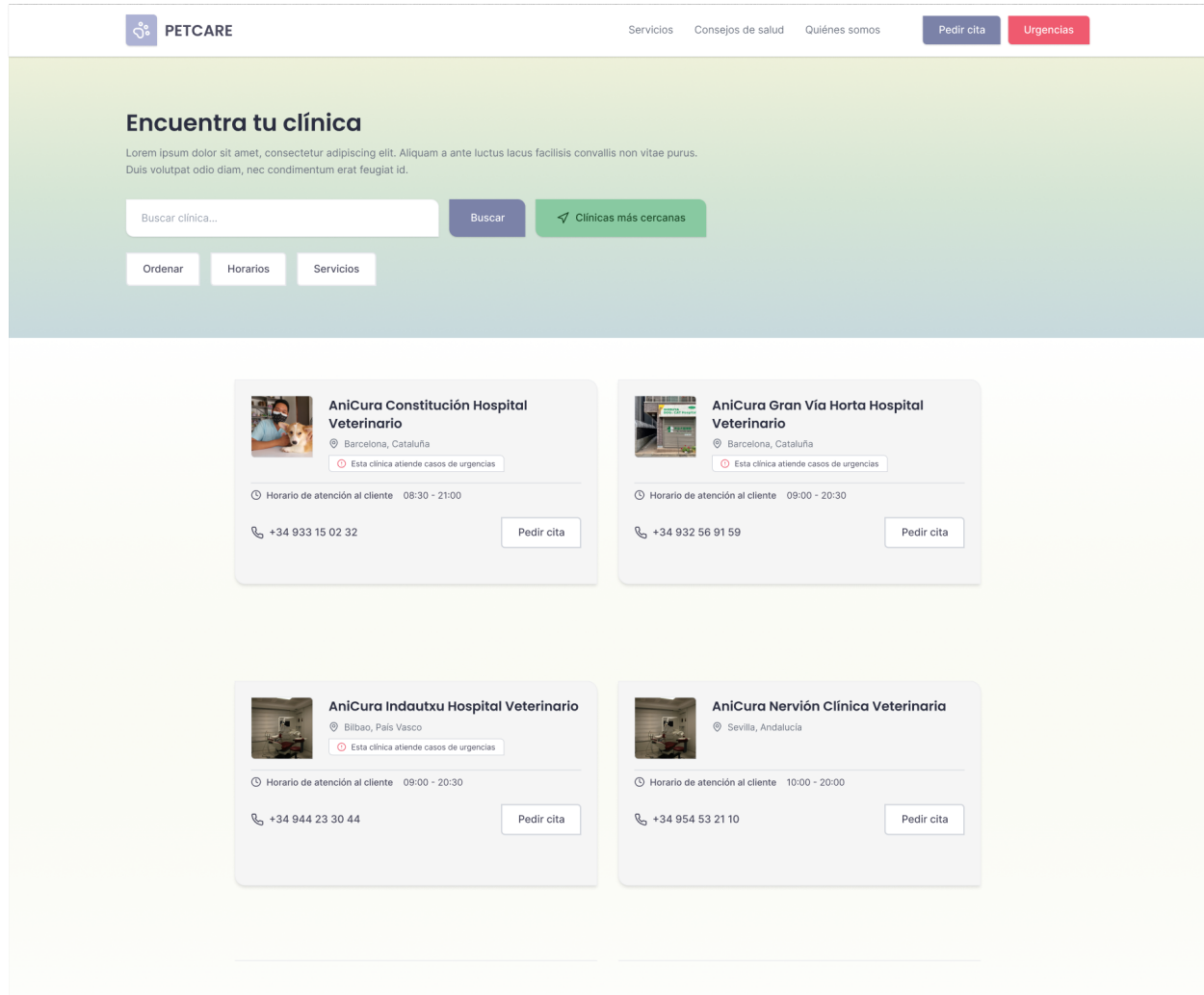


Figura 22: Mockup de página de clínicas

8.3.3. Urgencias

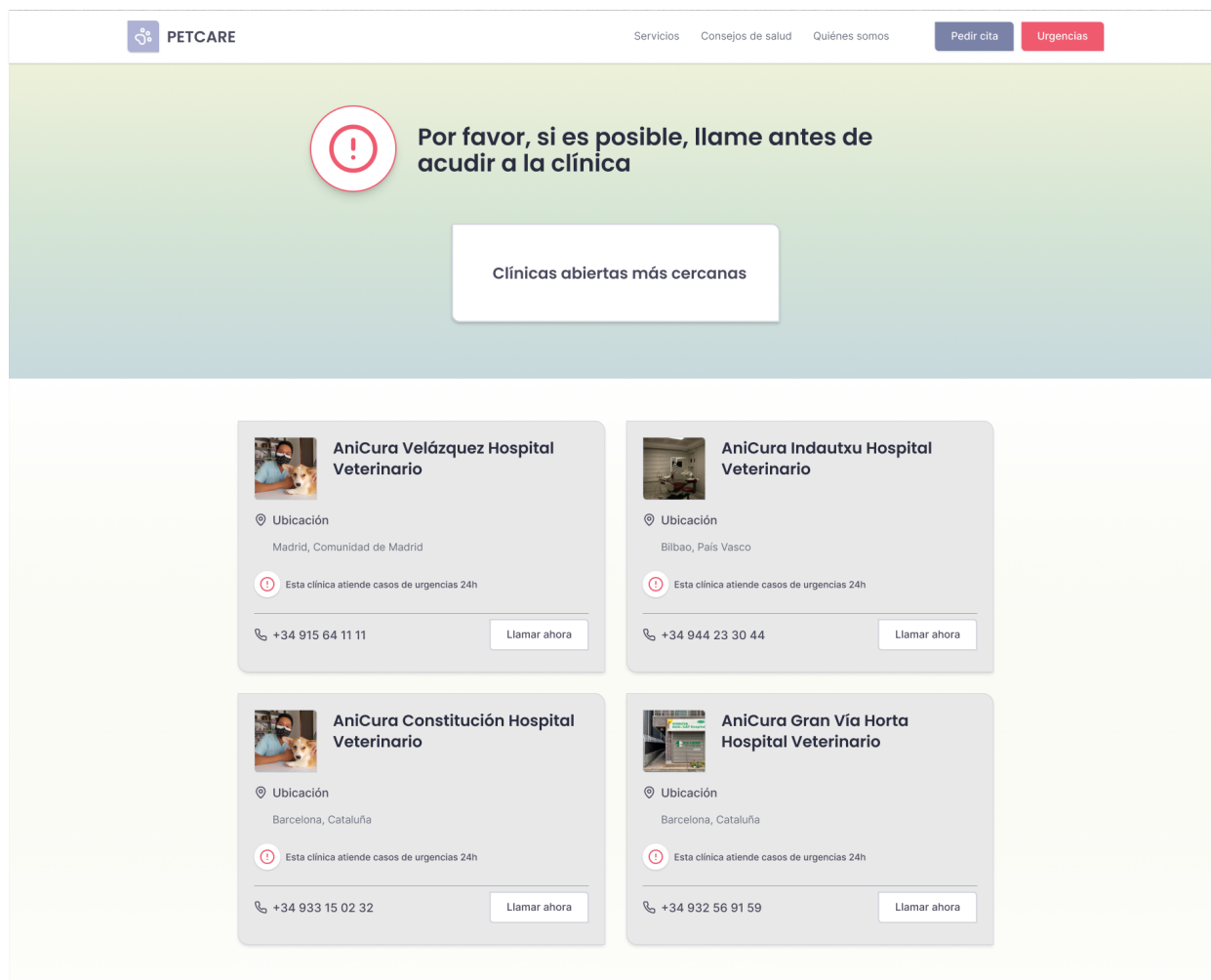


Figura 23: Mockup de página de urgencias

8.3.4. Servicios y tratamientos

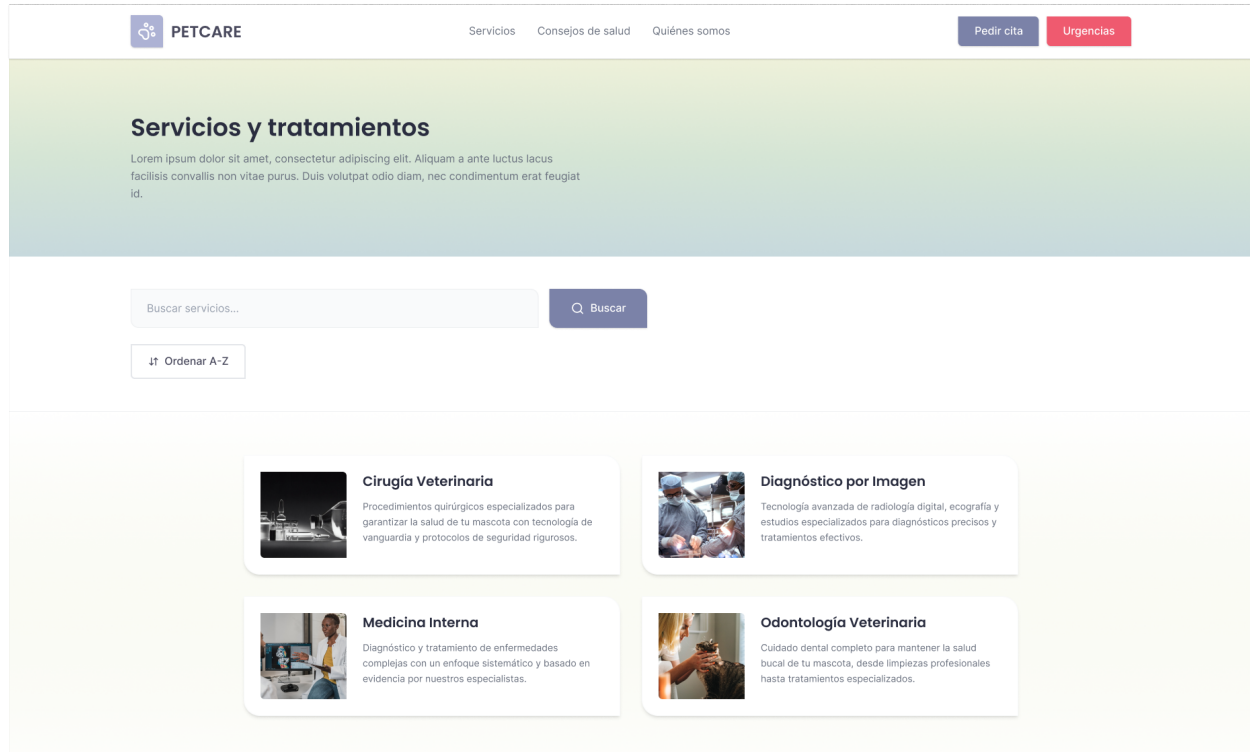


Figura 24: Mockup de página de servicios y tratamientos

8.3.5. Consejos de salud

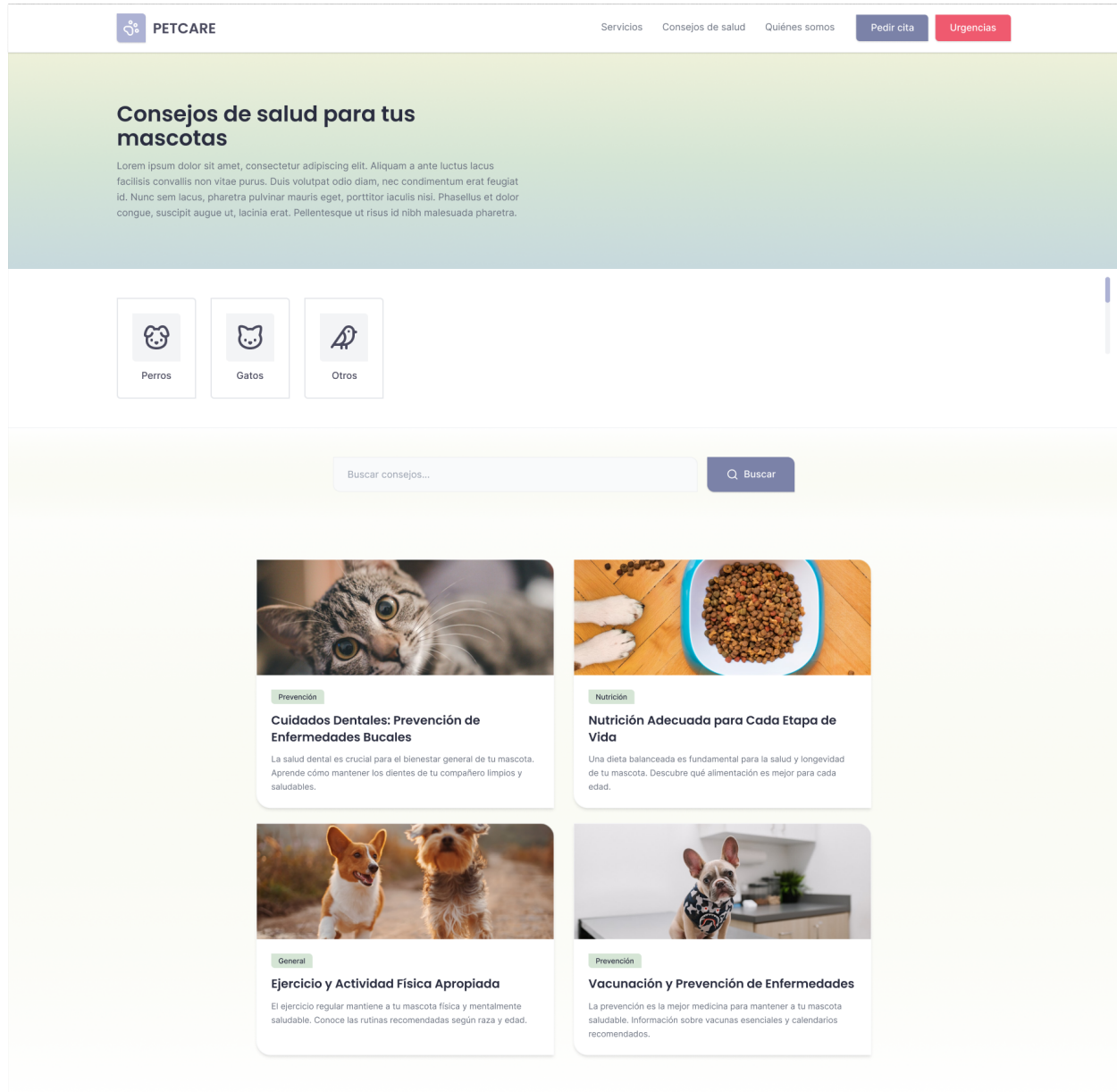


Figura 25: Mockup de la página de consejos de salud para mascotas

8.3.6. Sobre nosotros

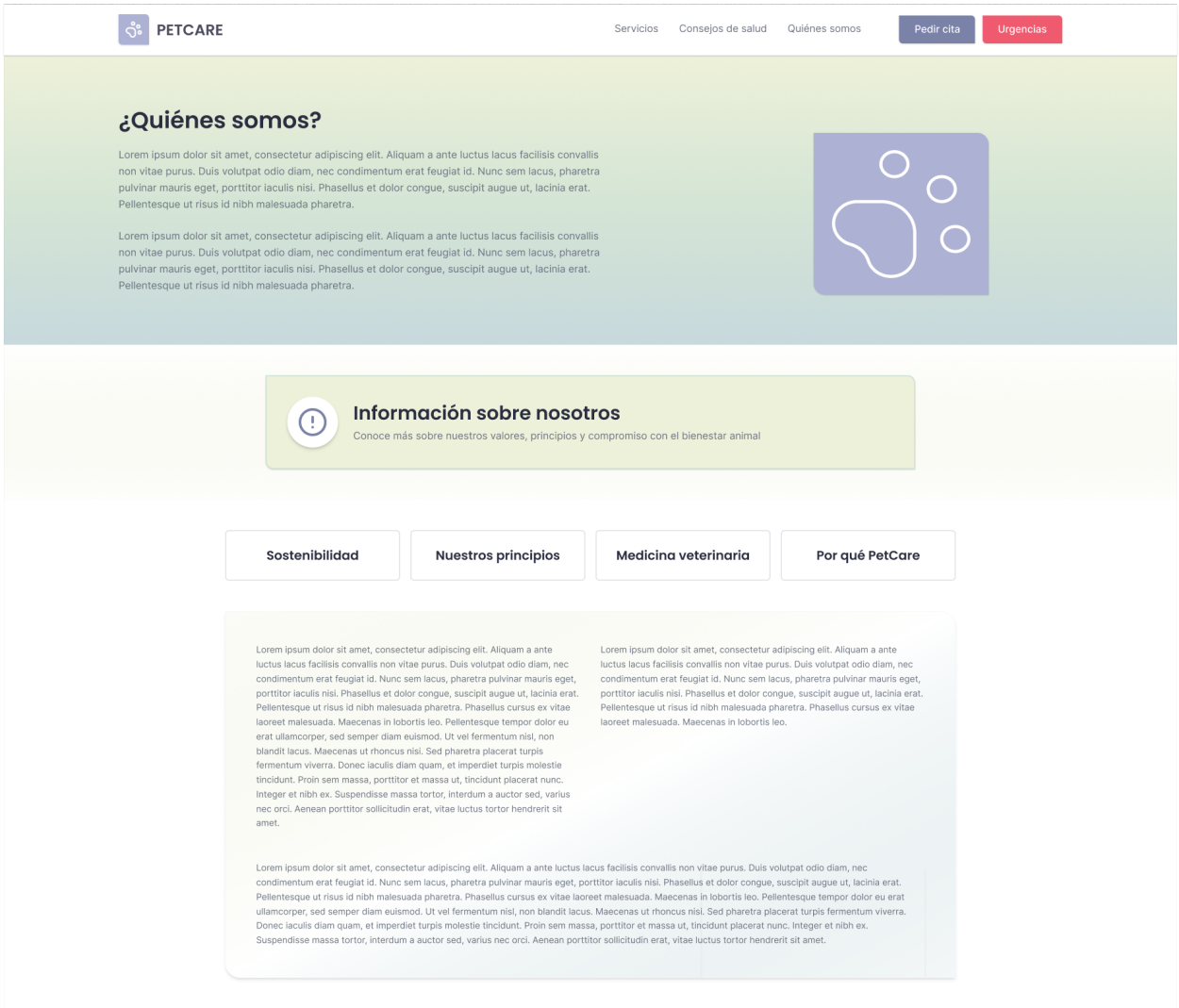


Figura 26: Mockup de página de información sobre la franquicia

8.3.7. Servicio específico

 PETCARE

ServiciosConsejos de saludQuiénes somosPedir citaUrgencias

[← Volver a Servicios](#)

Diagnóstico por Imagen

Tecnología avanzada para diagnósticos precisos y tratamientos efectivos



1. Radiología Digital

Contamos con equipos de radiología digital de última generación que nos permiten obtener imágenes de alta calidad en segundos. Esta tecnología reduce significativamente el tiempo de exposición de tu mascota a la radiación y proporciona resultados inmediatos.

Las radiografías digitales pueden ser ampliadas, mejoradas y compartidas fácilmente con especialistas cuando es necesario. Esto agiliza el proceso de diagnóstico y permite tomar decisiones terapéuticas más rápidas y precisas.

Utilizamos la radiología para diagnosticar una amplia variedad de condiciones, desde fracturas óseas hasta problemas cardíacos y respiratorios. Nuestro equipo de radiólogos veterinarios está altamente capacitado en la interpretación de imágenes complejas.

2. Ecografía Veterinaria

La ecografía es una herramienta diagnóstica no invasiva que nos permite visualizar los órganos internos en tiempo real. Es especialmente útil para evaluar el corazón, abdomen, sistema reproductivo y detectar masas o anomalías.

Nuestros ecógrafos de alta resolución proporcionan imágenes detalladas que ayudan a identificar problemas antes de que se conviertan en emergencias. El procedimiento es completamente indoloro y no requiere sedación en la mayoría de los casos.

Realizamos ecocardiografías especializadas para evaluar la función cardíaca, ecografías abdominales para problemas digestivos y urinarios, y estudios de gestación para monitorear el desarrollo de cachorros y gatitos.

3. Tomografía y Resonancia

Para casos complejos, contamos con acceso a servicios de tomografía computarizada (TC) y resonancia magnética (RM) a través de nuestra red de especialistas. Estas tecnologías proporcionan imágenes tridimensionales detalladas de estructuras internas.

La TC es particularmente útil para evaluar traumatismos, problemas neurológicos y planificar cirugías complejas. La RM es ideal para diagnosticar lesiones de tejidos blandos, problemas cerebrales y de médula espinal.

Coordinamos estos estudios especializados y trabajamos en conjunto con radiólogos veterinarios para interpretar los resultados y desarrollar el mejor plan de tratamiento para tu mascota.

¿Necesitas más información sobre este servicio?

Nuestro equipo está disponible para responder todas tus preguntas

[Encuentra tu clínica más cercana](#)

Figura 27: Mockup de la página de un servicio específico

8.3.8. Solicitud de cita

×

AniCura Constitución Hospital Veterinario

📍 Barcelona, Cataluña

Nombre completo

Teléfono

Correo electrónico

Nombre de la mascota

Tipo de mascota







Pedir cita

Figura 28: Mockup del formulario para pedir cita

9. Estructura de ficheros

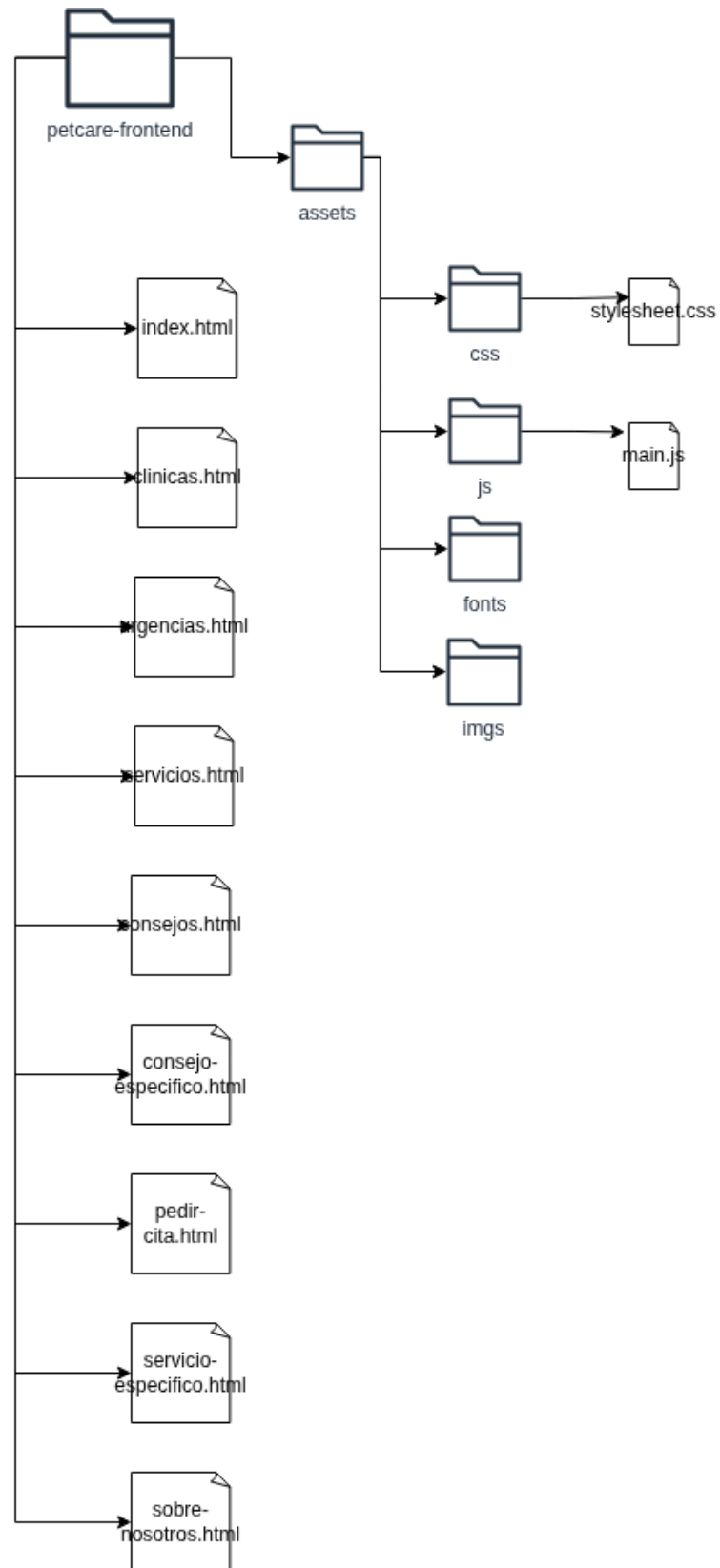


Figura 29: Estructura de ficheros

10. Mapas de etiquetas HTML

10.1. Página principal

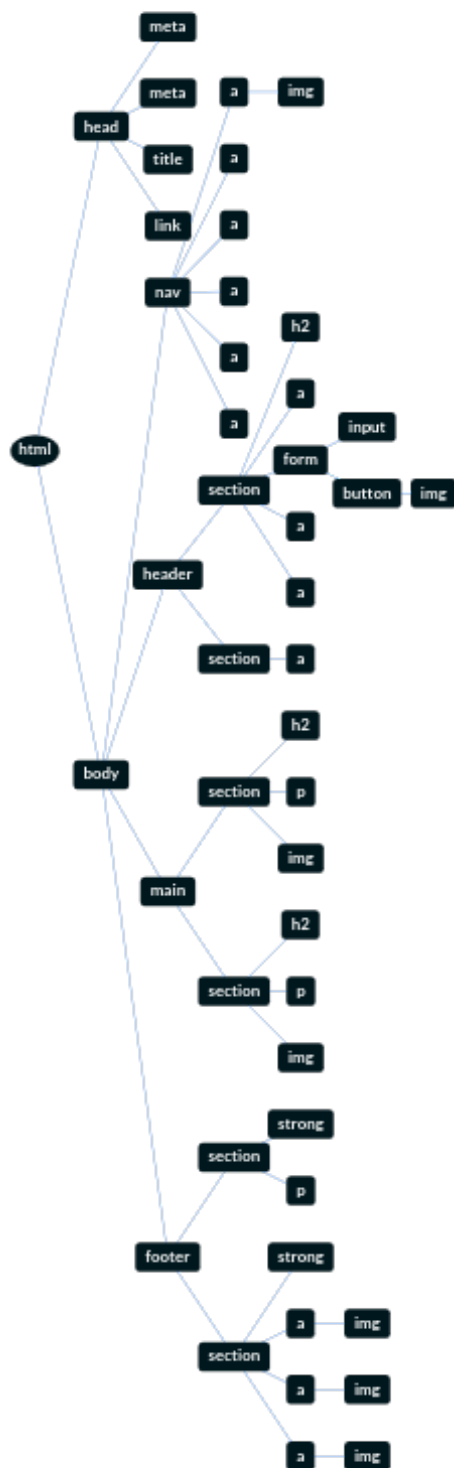


Figura 30: Mapa de etiquetas de la página principal

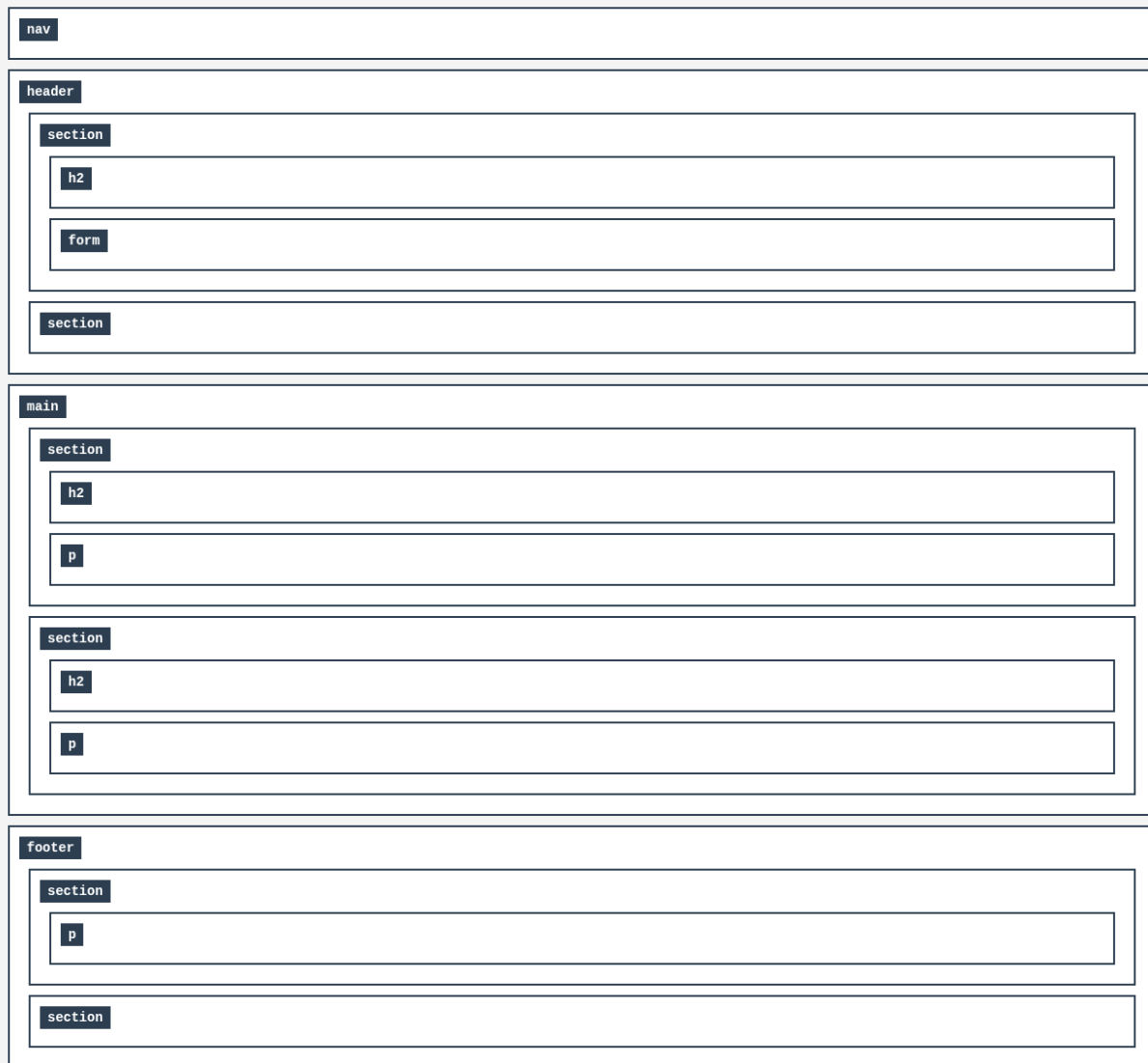


Figura 31: Mapa de etiquetas modelado en cajas de la página principal

10.2. Clínicas

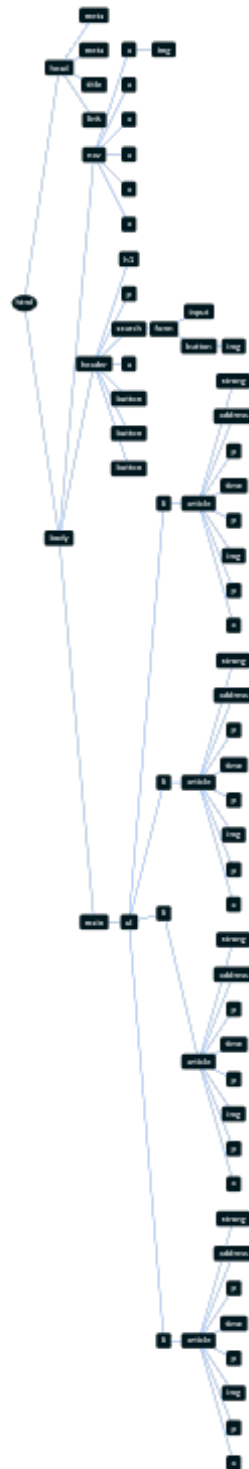


Figura 32: Mapa de etiquetas de la página de clínicas

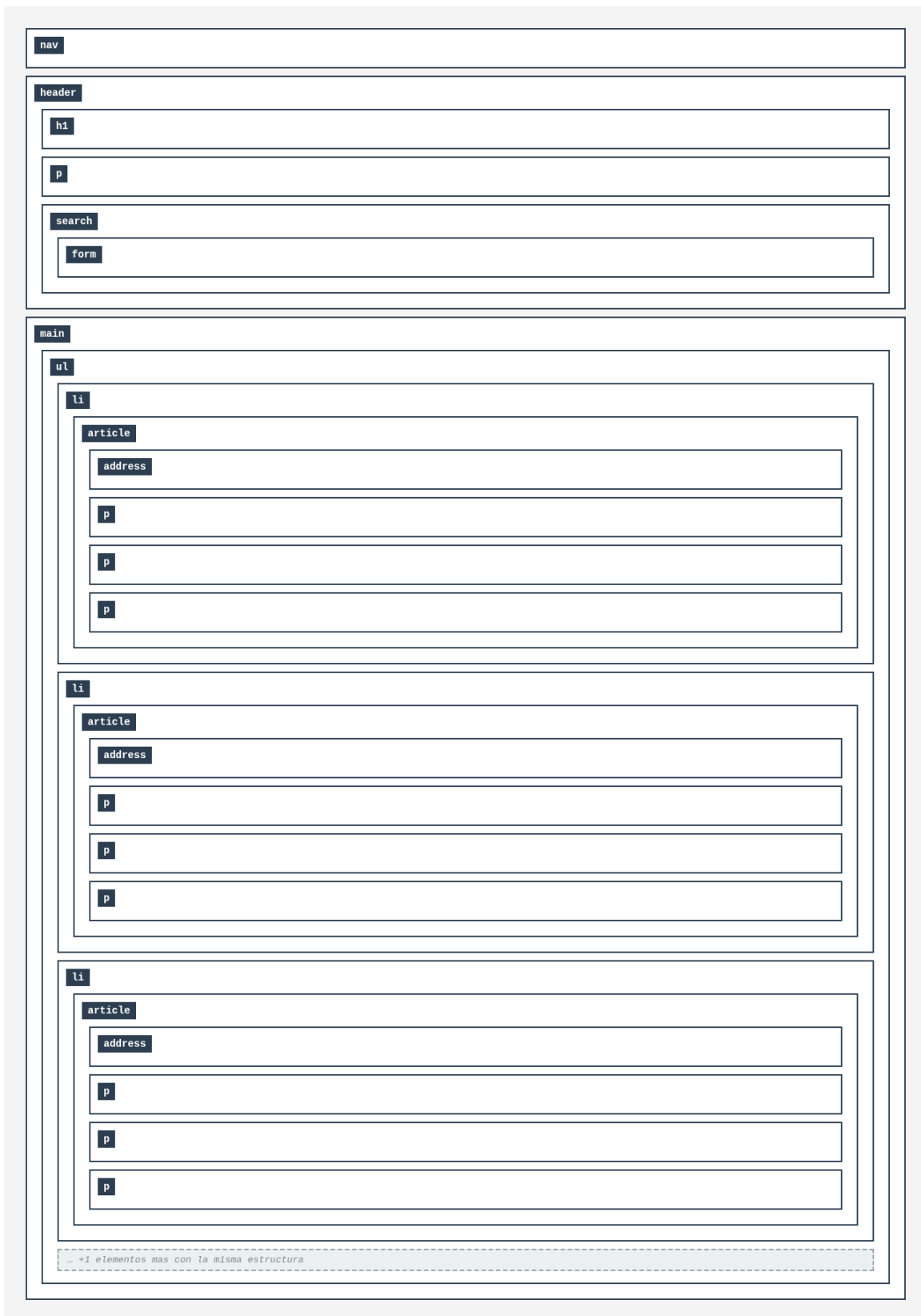


Figura 33: Mapa de etiquetas modelado en cajas de la página de clínicas

10.3. Urgencias

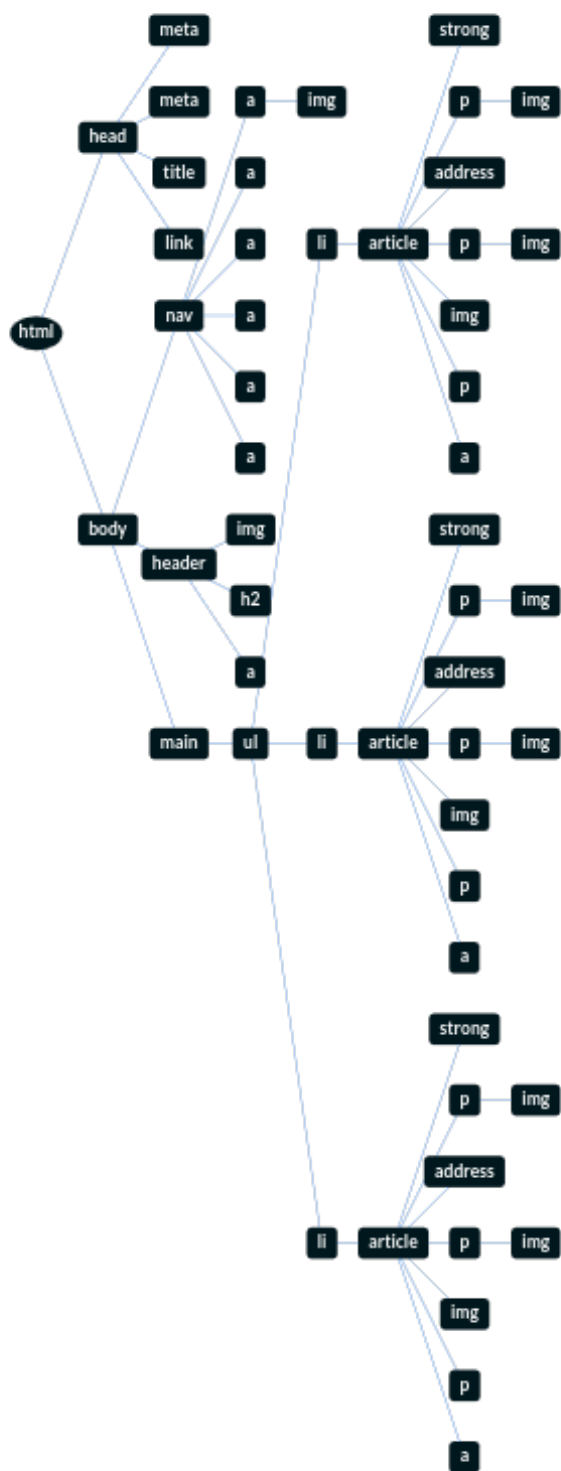


Figura 34: Mapa de etiquetas de la página de urgencias



Figura 35: Mapa de etiquetas modelado en cajas de la página de urgencias

10.4. Servicios y tratamientos

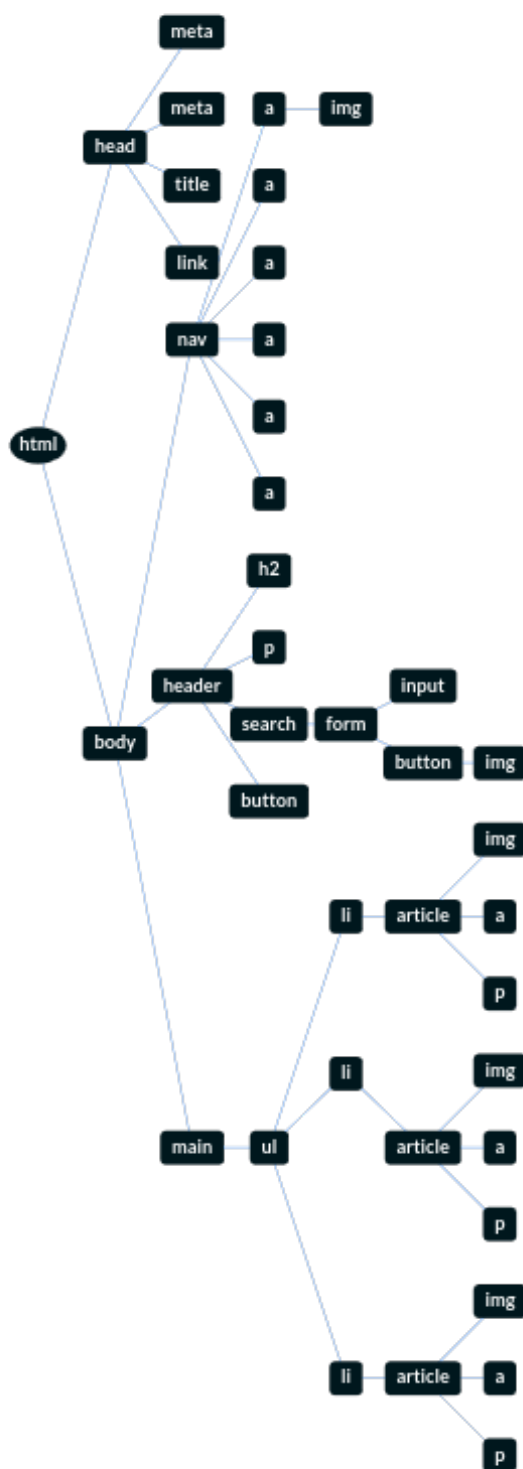


Figura 36: Mapa de etiquetas de la página de servicios y tratamientos



Figura 37: Mapa de etiquetas modelado en cajas de la página de servicios y tratamientos

10.5. Servicio específico

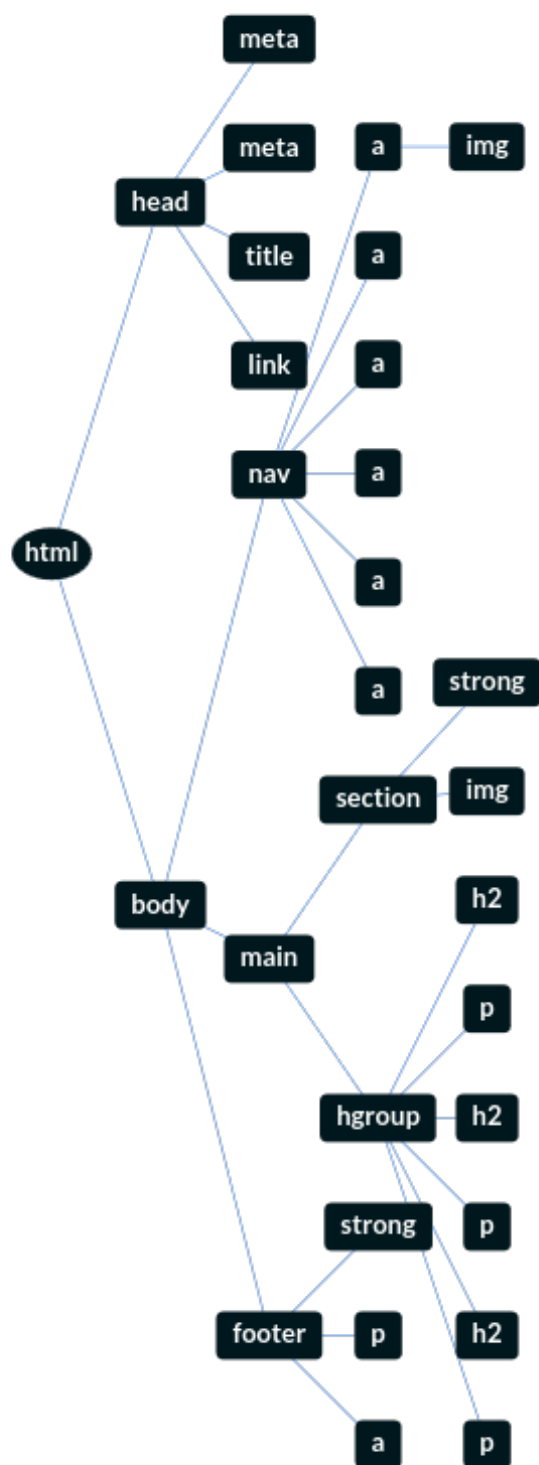


Figura 38: Mapa de etiquetas de la página de un servicio específico



Figura 39: Mapa de etiquetas modelado en cajas de la página de un servicio específico

10.6. Sobre nosotros

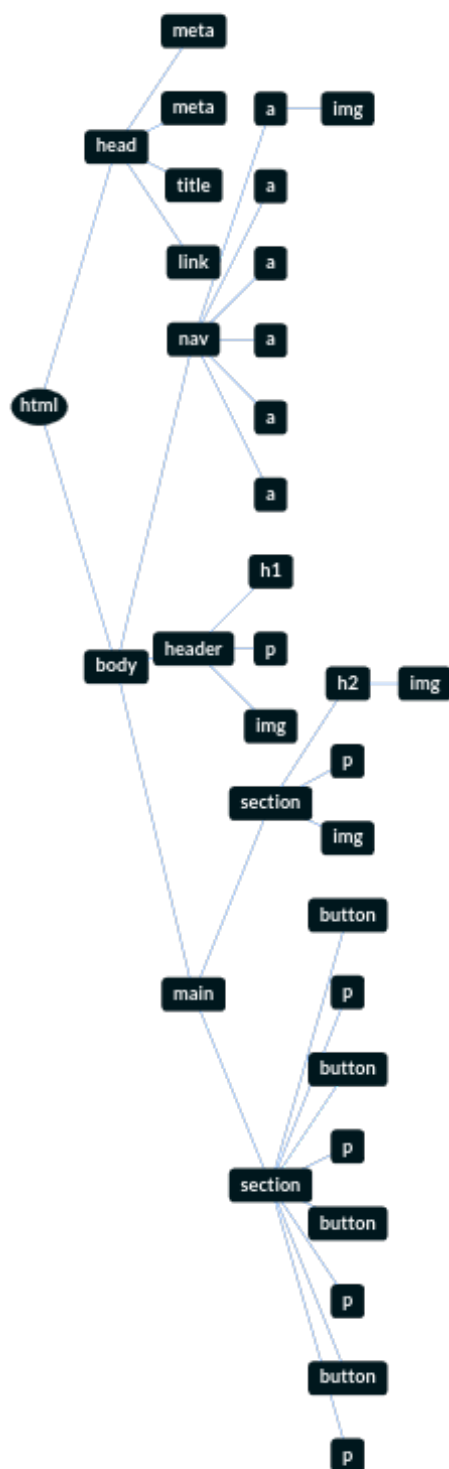


Figura 40: Mapa de etiquetas de la página de información sobre la franquicia



Figura 41: Mapa de etiquetas modelado en cajas de la página de información sobre la franquicia

10.7. Consejos de salud

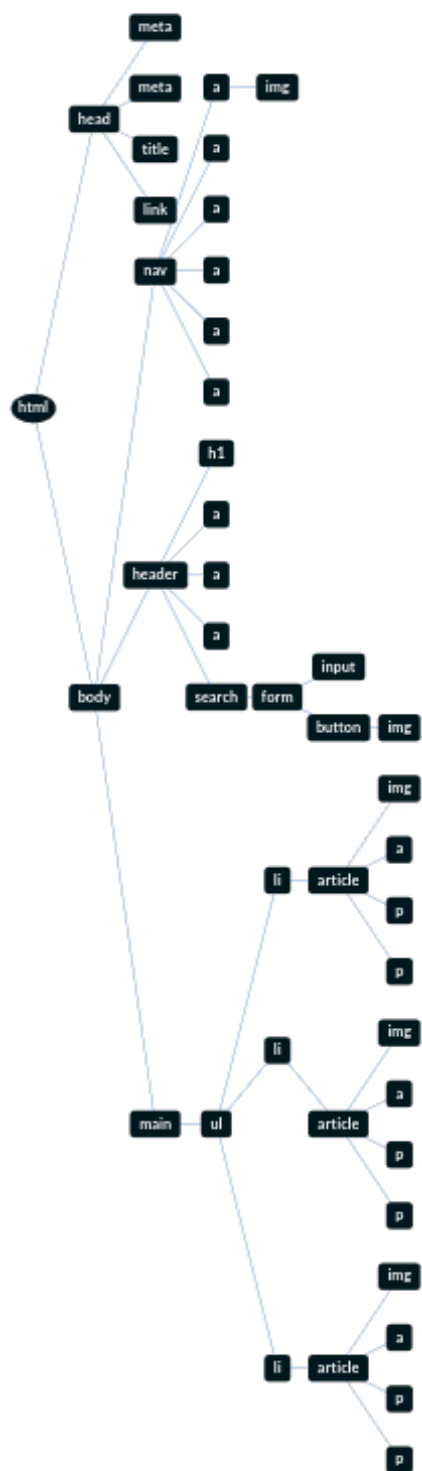


Figura 42: Mapa de etiquetas de la página de consejos de salud



Figura 43: Mapa de etiquetas modelado en cajas de la página de consejos de salud

10.8. Consejo específico

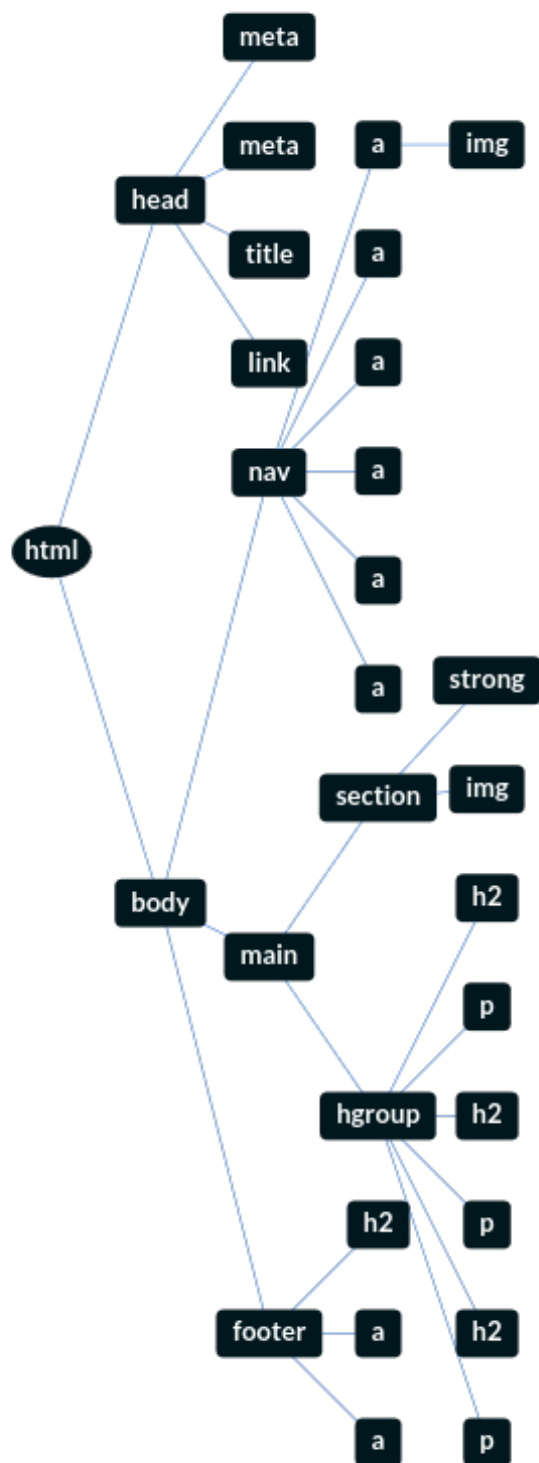


Figura 44: Mapa de etiquetas de la página de un consejo específico



Figura 45: Mapa de etiquetas modelado en cajas de la página de un consejo específico

10.9. Solicitud de cita

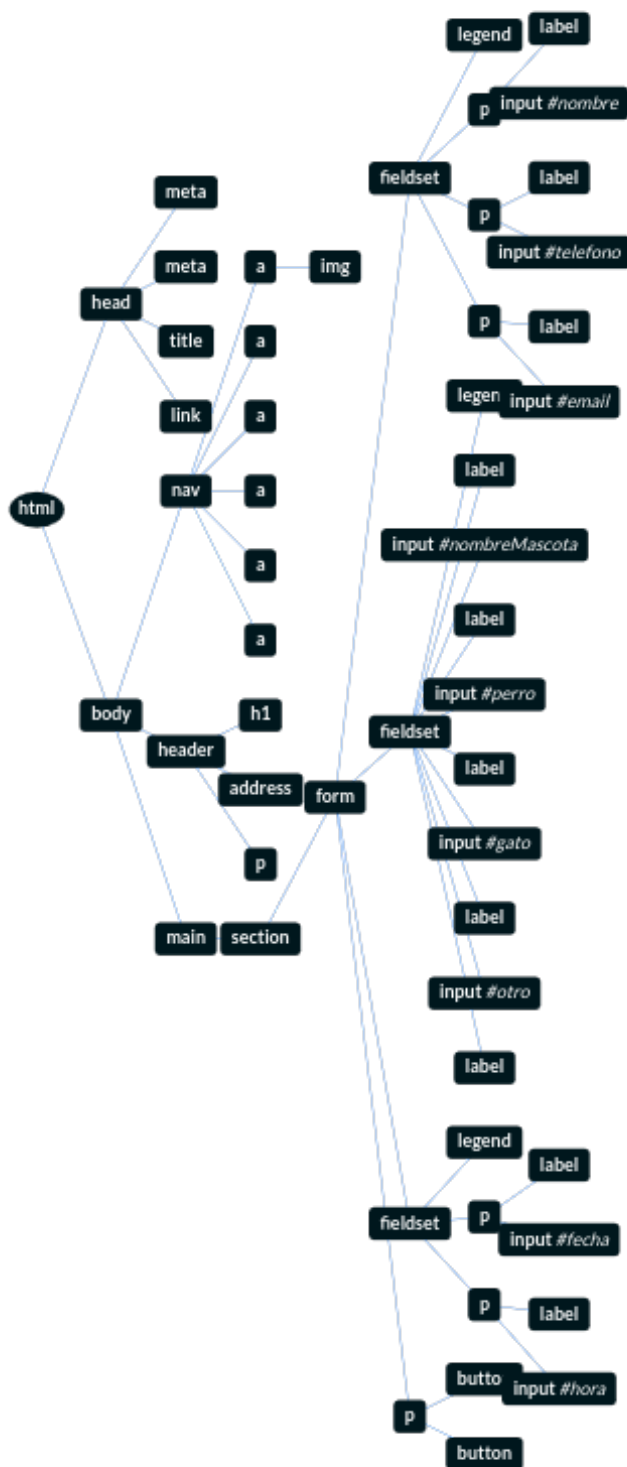


Figura 46: Mapa de etiquetas del formulario para pedir cita

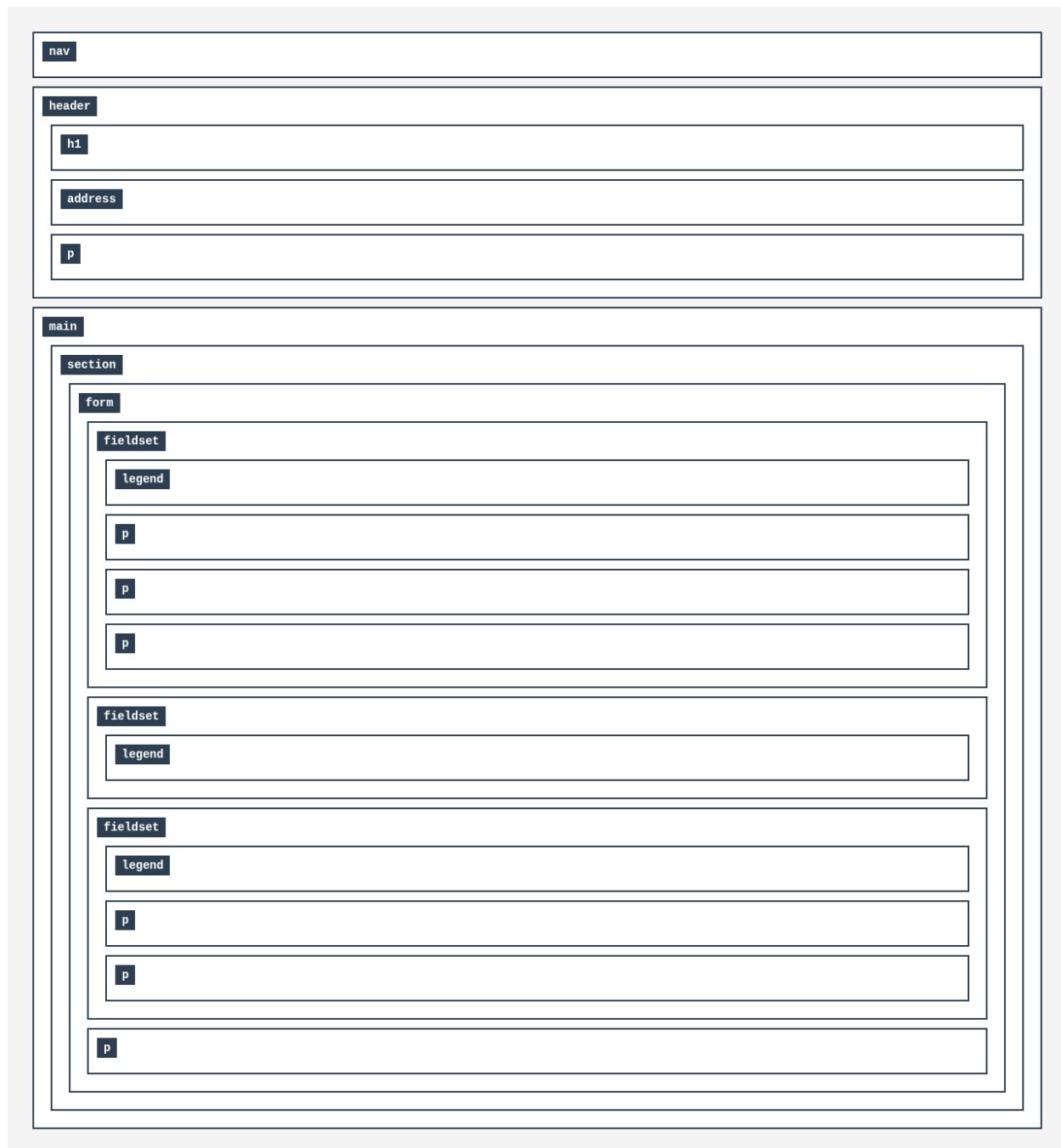


Figura 47: Mapa de etiquetas modelado en cajas del formulario para pedir cita

11. Práctica 3: Responsividad

En esta práctica se ha dotado al sitio web de un diseño responsivo aplicando cinco técnicas distintas, cada una en una página diferente, tal y como indica el enunciado. En todos los casos se sigue una filosofía *Mobile First*, priorizando la visualización en dispositivos pequeños y escalando hacia pantallas mayores mediante *Media Queries*.

11.1. Flexible Grids con float — Clínicas

En la sección de clínicas, hemos implementado el patrón de diseño *Flexible Grids* apoyado en el uso de elementos flotantes (`float`). Esta decisión se enmarca dentro de una filosofía *Mobile First*, priorizando el rendimiento y la correcta visualización en dispositivos móviles por defecto, para posteriormente escalar el diseño hacia pantallas de mayor tamaño.

Por defecto (para pantallas pequeñas), las tarjetas de las clínicas se comportan como elementos de bloque estándar, ocupando el 100 % del ancho disponible y apilándose verticalmente en una única columna.

Sin embargo, cuando el ancho de la ventana supera nuestro punto de quiebre de 992 píxeles (dispositivos de escritorio), entra en acción nuestra *Media Query*. En este punto, el contenedor altera el comportamiento de sus elementos hijos (`<li>`) de la siguiente manera:

- **Uso de unidades relativas:** Se abandona el ancho del 100 % y se asigna a cada tarjeta un `width` del 47 %, un tamaño relativo que permite que el diseño fluya y se estire proporcionalmente al tamaño del contenedor padre.
- **Flujos opuestos:** Utilizamos el pseudo-selector `:nth-child` para distinguir entre elementos pares e impares. Los elementos impares flotan a la izquierda (`float: left`) con un margen derecho del 6 % (completando así el 100 % del ancho disponible: 47 % + 6 % + 47 %). Los elementos pares flotan a la derecha (`float: right`).
- **Limpieza de floats:** Para evitar que las tarjetas de diferentes filas colisionen o se solapen debido a diferencias de altura, aplicamos la propiedad `clear: both` a los elementos impares, forzando un salto de línea limpio para cada nueva fila del *grid*.

A continuación, se detalla el código CSS utilizado para lograr este comportamiento:

```
1 .contenedor-tarjetas {
2   list-style: none;
3   width: 100%;
4   overflow: hidden; /* Limpia el desbordamiento de los floats hijos */
5   margin: auto;
6 }
7 /* Comportamiento por defecto (Mobile First) */
8 .contenedor-tarjetas li {
9   width: 100%;
10  float: none;
11  margin: 20px auto;
12 }
13 /* Escalado a pantallas grandes (Desktop) */
14 @media screen and (min-width: 992px) {
15   .contenedor-tarjetas {
16     margin-top: 2rem;
17     margin-bottom: 5rem;
18     padding: 2rem;
19   }
20   .contenedor-tarjetas li {
21     width: 47%;
22     margin-bottom: 2rem;
23   }
24   /* Tarjetas de la izquierda */
25   .contenedor-tarjetas li:nth-child(odd) {
```

```

26     float: left;
27     margin-right: 6%;
28     clear: both; /* Fuerza el inicio de una nueva fila */
29 }
30 /* Tarjetas de la derecha */
31 .contenedor-tarjetas li:nth-child(even) {
32     float: right;
33     margin-right: 0;
34 }
35 }

```

Listing 1: Implementación de Flexible Grids para las tarjetas de clínicas

Las siguientes capturas muestran el comportamiento responsivo de la página de clínicas en escritorio y en móvil:

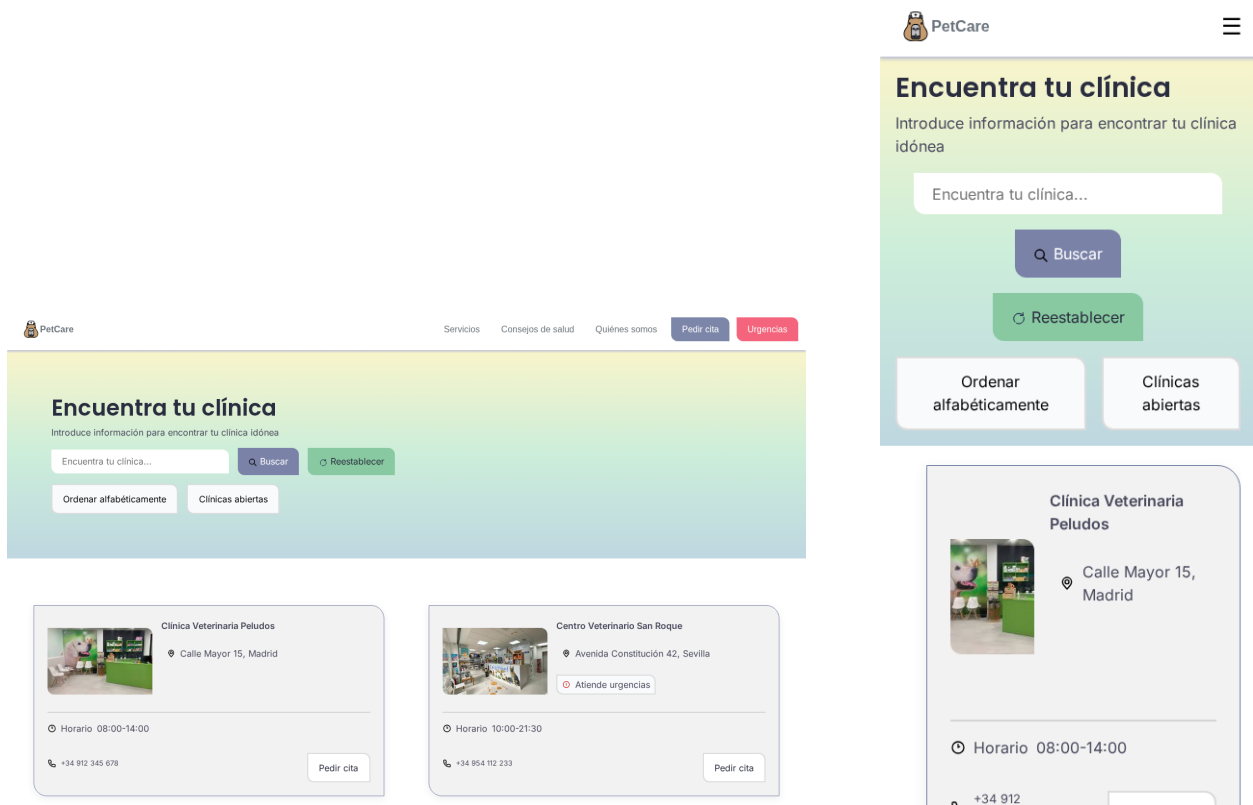


Figura 48: Clínicas: versión escritorio (izquierda) y móvil (derecha)

El mapa de etiquetas actualizado (Práctica 3) refleja los cambios introducidos respecto a la versión anterior de la página:

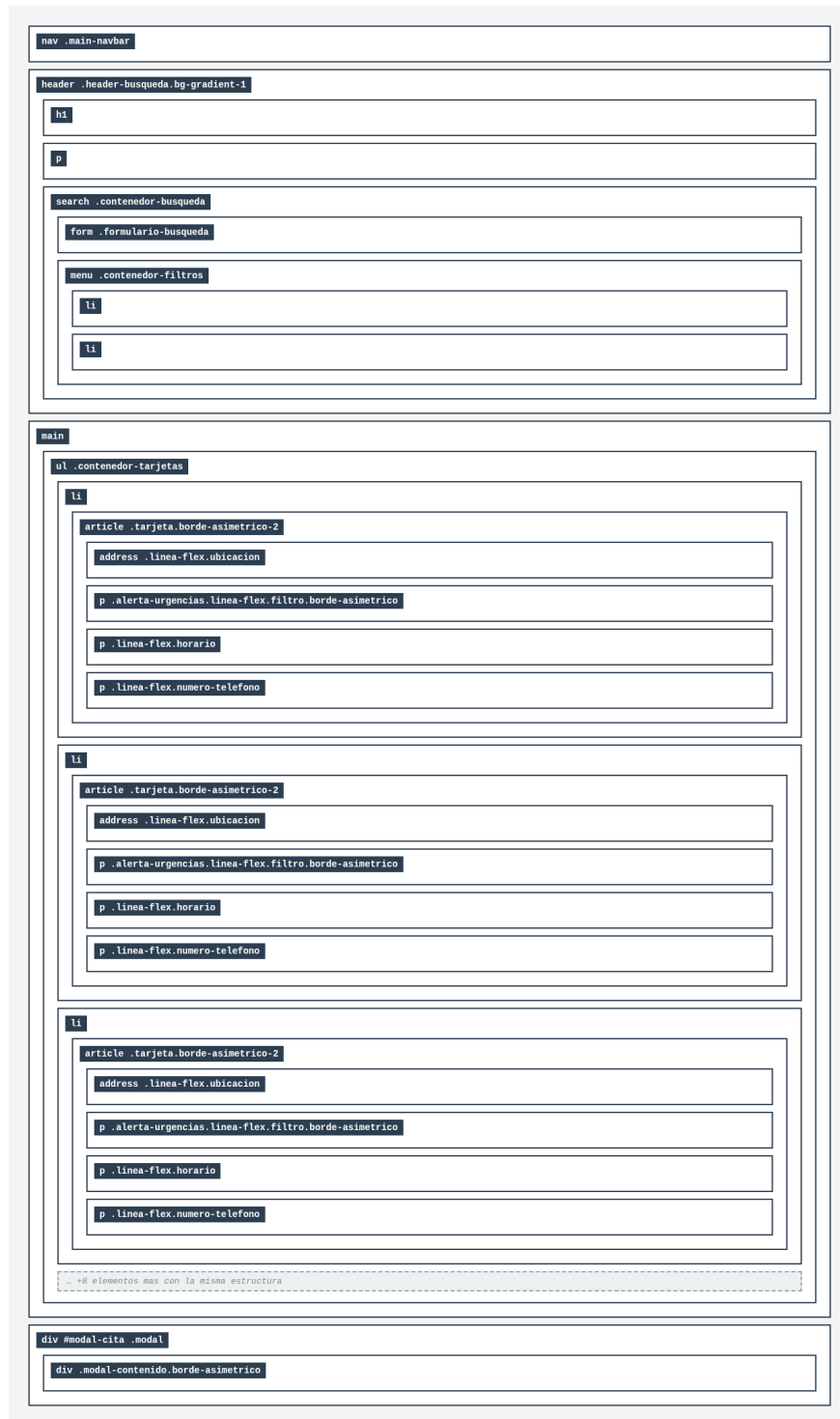


Figura 49: Mapa de etiquetas actualizado (P3) — Clínicas

11.2. Diseño multicolumna — Sobre nosotros

En la página “Sobre nosotros”, los apartados de información extensa (Sostenibilidad, Nuestros principios, Medicina veterinaria y Por qué PetCare) requieren una estructura que facilite su lectura. Para ello, hemos utilizado las propiedades nativas del módulo *Multi-column Layout* de CSS3, logrando que el texto fluya de forma automática en una o dos columnas dependiendo del dispositivo.

En monitores de escritorio, mostrar un bloque de texto que cruce la pantalla de lado a lado resulta fatigante para el ojo humano. Dividir el contenido en dos columnas (como en un periódico) acorta la longitud de las líneas y mejora la legibilidad. Sin embargo, en dispositivos móviles o tabletas pequeñas, mantener dos columnas provocaría que el texto quedara demasiado comprimido y las palabras se cortaran en exceso, por lo que la lectura debe volver a una sola columna.

Para este módulo específico, el código base establece el diseño para pantallas grandes y utilizamos una *Media Query* con `max-width` para adaptar el contenido cuando la pantalla se reduce.

- **Diseño base (Pantallas grandes):** Asignamos al contenedor la propiedad `column-count: 2`, que le indica al navegador que divida el texto en dos columnas equitativas de forma automática. Para que el texto no se vea demasiado compacto, utilizamos `column-gap: 2rem` para establecer algo de separación entre ambas columnas. El contenedor ocupa un 75 % del ancho de la pantalla para mantener unos márgenes laterales.
- **Punto de quiebre (Dispositivos móviles):** Cuando la pantalla es inferior a 992 píxeles, la regla `@media screen and (max-width: 992px)` se activa. Reducimos el número de columnas a una sola (`column-count: 1`). Además, ampliamos el contenedor al 95 % del ancho de la pantalla para aprovechar al máximo el espacio reducido de los teléfonos, y reducimos ligeramente el tamaño de la fuente (de `x-large` a `large`) para adaptarlo a la nueva resolución.

A continuación, se detalla el código CSS correspondiente a esta implementación:

```
1  /* Diseño base para pantallas grandes (Escritorio) */
2  .contenedor-multicol {
3      column-width: 50%;
4      column-count: 2; /* Divide el texto en dos columnas */
5      column-gap: 2rem; /* Espacio de respiracion entre columnas */
6      width: 75%;
7      margin: auto;
8      padding: 35px;
9      background: linear-gradient(0deg,
10     rgba(113, 212, 154, 0.15) 50%,
11     rgba(145, 210, 172, 0.15) 50%,
12     rgba(255, 255, 255, 0.15) 100%);
13     font-size: x-large;
14 }
15 /* Adaptacion para dispositivos moviles y tabletas */
16 @media screen and (max-width: 992px) {
17     .contenedor-multicol {
18         column-width: 50%;
19         column-count: 1; /* Retorna a una unica columna vertical */
20         column-gap: 2rem;
21         width: 95%;
22         margin: auto;
23         padding: 35px;
24         background: linear-gradient(0deg,
```

```

25     rgba(113, 212, 154, 0.15) 50%,
26     rgba(145, 210, 172, 0.15) 50%,
27     rgba(255, 255, 255, 0.15) 100%);
28     font-size: large;
29 }
30 }

```

Listing 2: Uso de Multi-column Layout para la adaptación de textos largos

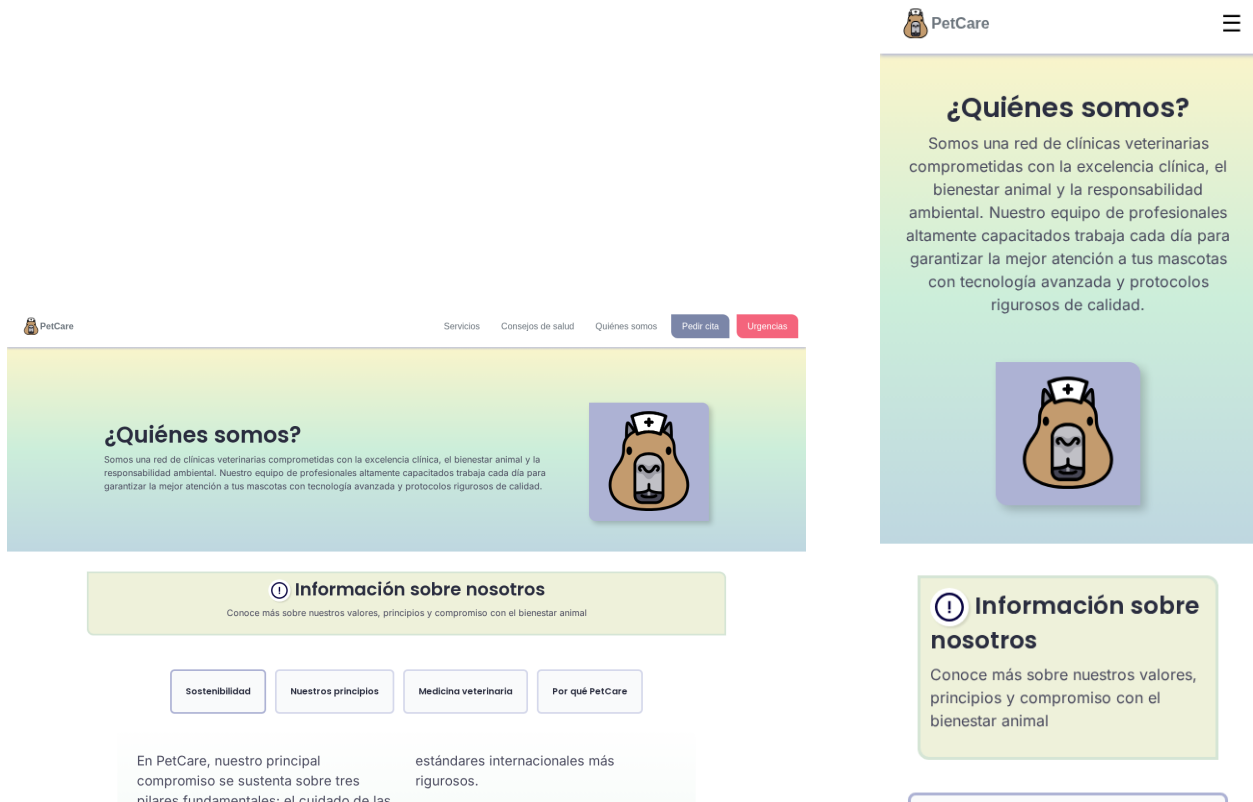


Figura 50: Sobre nosotros: versión escritorio (izquierda) y móvil (derecha)



Figura 51: Mapa de etiquetas actualizado (P3) — Sobre nosotros

11.3. CSS Flex Container — Solicitud de cita

Para la página de solicitud de cita (que se abre como un `iframe` al hacer clic en el botón de pedir cita de la tarjeta de una clínica) hemos decidido utilizar el módulo *CSS Flexbox*. A diferencia de CSS Grid, que es ideal para la estructura general de la página (pues permite la distribución bidimensional de elementos), Flexbox brilla en la alineación y distribución de elementos en una sola dimensión (filas o columnas).

Hemos aplicado Flexbox a diferentes niveles de profundidad dentro del formulario para lograr una estructura sólida e intuitiva:

- **Centrado global (El contenedor Body):** Convertimos el `body` del `iframe` en un contenedor flex en dirección columna. Al sumarle `min-height: 100vh` y `justify-content: center`, logramos que todo el formulario quede perfectamente centrado verticalmente dentro de la ventana modal, independientemente del tamaño de la pantalla.
- **Apilamiento lógico (Columnas):** Los contenedores principales como `.fieldset-propietario` y `.fieldset-mascota` utilizan `flex-direction: column`. Esto asegura que los campos de introducción de datos se apilen de arriba hacia abajo, el comportamiento natural y más accesible para rellenar un formulario.
- **Agrupaciones horizontales (Filas):** Para optimizar el espacio, agrupamos elementos estrechos y relacionados entre sí usando `flex-direction: row`. Esto lo vemos en `#datos-contacto` (teléfono y email), `.opciones-especie` (botones de radio para el tipo de mascota) y `.fieldset-cita` (fecha y hora). Aplicando `justify-content: space-between`, garantizamos que estos elementos se separen uniformemente ocupando todo el ancho disponible.
- **El botón de envío:** Al botón principal (`.boton-submit`) le asignamos un `width: 100%`. En un contexto flexible (y asumiendo que el contenedor padre permite el `flex-wrap`), forzar un ancho del 100 % empuja al botón a ocupar su propia línea independiente al final del formulario.

A continuación, se detalla el código CSS estructural que define este comportamiento:

```
1  /* Centrado vertical de todo el formulario en el modal */
2  body {
3    display: flex;
4    flex-direction: column;
5    justify-content: center;
6    min-height: 100vh;
7    margin: 0;
8  }
9  /* Apilamiento vertical para los bloques principales */
10 .fieldset-propietario, .fieldset-mascota {
11   display: flex;
12   flex-direction: column;
13   gap: 0.5rem;
14 }
15 /* Distribucion en fila para campos complementarios */
16 #datos-contacto {
17   display: flex;
18   flex-direction: row;
19   width: 100%;
20   gap: 0.5rem;
21 }
22 /* Distribucion con separacion equitativa para fechas y opciones */
```

```

23 .opciones-especie, .fieldset-cita {
24     display: flex;
25     flex-direction: row;
26     justify-content: space-between;
27     gap: 0.5rem;
28 }
29 .opciones-especie {
30     padding: 1rem;
31 }
32 /* Boton de envio forzado a ocupar una linea completa */
33 .boton-submit {
34     background-color: var(--color-cta-primary);
35     color: white;
36     border: none;
37     padding: 1rem 0rem;
38     font-size: 1.5rem;
39     font-weight: bold;
40     width: 100%;
41 }

```

Listing 3: Uso de CSS Flexbox para la maquetación del formulario de citas

The image displays two versions of a web form for requesting a clinic appointment. The left version is the desktop layout, and the right version is the mobile layout.

Desktop Version (Left):

- Header:** "Nombre de la clínica" with a location pin icon and "Ubicación de la clínica".
- Form Fields:**
 - A single wide input for "Nombre completo".
 - Two side-by-side inputs for "Número de teléfono" and "Correo electrónico".
 - A single wide input for "Nombre de la mascota".
- Mascota Type:** A label "Tipo de mascota" followed by three square buttons with icons for a dog, a cat, and a bird.
- Appointment Time:** Two labels, "Fecha de la cita:" and "Hora de la cita:", each followed by a date/time picker interface.
- Submit Button:** A wide blue button at the bottom labeled "Pedir cita".

Mobile Version (Right):

- Header:** "Nombre de la clínica" with a location pin icon and "Ubicación de la clínica".
- Form Fields:**
 - A single input for "Nombre completo".
 - Two stacked inputs for "Número de teléfono" and "Correo electrónico".
 - A single input for "Nombre de la mascota".
- Mascota Type:** A label "Tipo de mascota" followed by three square buttons with icons for a dog, a cat, and a bird.
- Appointment Time:** Two labels, "Fecha de la cita:" and "Hora de la cita:", each followed by a date/time picker interface.
- Submit Button:** A blue button labeled "Pedir cita".

Figura 52: Solicitud de cita: versión escritorio (izquierda) y móvil (derecha)



Figura 53: Mapa de etiquetas actualizado (P3) — Solicitud de cita

11.4. CSS Grid — Servicios

En la página de servicios, hemos optado por emplear el módulo *CSS Grid Layout* para organizar la presentación de los distintos servicios ofrecidos por las clínicas. Al igual que en la estructura de las tarjetas anteriores (las tarjetas de clínicas), esta maquetación se ha desarrollado siguiendo estrictamente la metodología *Mobile First*.

CSS Grid es una herramienta moderna para manejar diseños bidimensionales. A diferencia de `float` o el *Multi-column Layout*, Grid nos permite definir una cuadrícula rígida pero elástica, asegurando que todas las cajas de servicios mantengan anchos uniformes y un espaciado adecuado sin necesidad de calcular márgenes porcentuales. En dispositivos móviles, una sola columna garantiza legibilidad y un tamaño adecuado para la interacción táctil. En pantallas de mayor tamaño, dividir el contenido en dos columnas evita que las cajas se estiren de forma antiestética, mejorando el escaneo visual de la información.

La implementación se articula definiendo el estado base para pantallas pequeñas y aplicando una *Media Query* para el escalado en pantallas medianas y de escritorio:

- **Diseño base (Dispositivos móviles):** Convertimos el contenedor de la lista (`.lista-servicios`) en una cuadrícula aplicando `display: grid`. A través de la propiedad `grid-template-columns: 1fr`, ordenamos al navegador que cree una única columna que ocupe `1fr` (una fracción), lo que se traduce en el 100 % del espacio disponible. La propiedad `gap: 1rem` se encarga de aplicar un margen uniforme entre todos los servicios sin ensuciar el código con márgenes individuales.
- **Punto de quiebre (A partir de 768px):** Cuando la pantalla alcanza el tamaño típico de una tablet en vertical (768 píxeles) o superior, entra en acción la regla `@media` (`min-width: 768px`). En este punto, reescribimos la estructura ordenando `grid-template-columns: 1fr 1fr`. Esta sencilla línea recalcula todo el diseño para formar dos columnas de proporciones exactamente iguales. Además, incrementamos la separación a `gap: 1.25rem` para proporcionar más respiro visual al contenido en pantallas amplias.

A continuación, se presenta el bloque de código CSS que gobierna este comportamiento:

```
1  /* Comportamiento por defecto (Mobile First) */
2  .lista-servicios {
3    list-style: none;
4    margin: 0;
5    padding: 0;
6    display: grid;
7    grid-template-columns: 1fr; /* Una unica columna que ocupa todo el
8                                espacio */
9    gap: 1rem;
10 }
11 /* Escalado para tablets y pantallas de escritorio */
12 @media (min-width: 768px) {
13   .lista-servicios {
14     /* Divide el espacio disponible en dos columnas simetricas */
15     grid-template-columns: 1fr 1fr;
16     gap: 1.25rem;
17   }
18 }
```

Listing 4: Implementación de Mobile First mediante CSS Grid

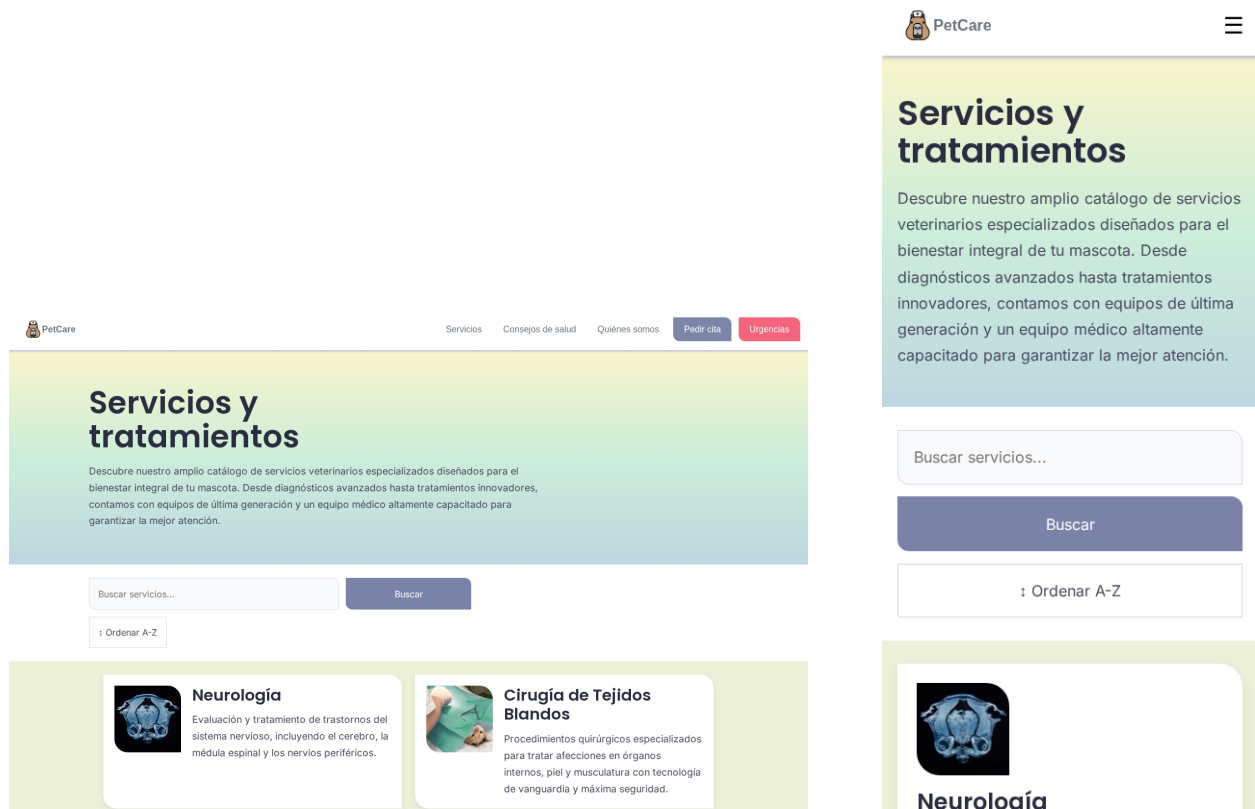


Figura 54: Servicios: versión escritorio (izquierda) y móvil (derecha)



Figura 55: Mapa de etiquetas actualizado (P3) — Servicios

11.5. Framework Bootstrap — Consejos de salud

En la página de consejos de salud, hemos complementado con **Bootstrap**, uno de los *frameworks* de CSS más populares en el desarrollo web. Un *framework* como este nos proporciona una biblioteca de estilos predefinidos y un sistema de cuadrículas robusto basado en clases de utilidad, lo que permite construir interfaces responsivas de forma mucho más ágil.

Mientras que en otras páginas hemos desarrollado nuestras propias reglas CSS desde cero, Bootstrap nos permite implementar estos componentes rápidamente y asegurar su compatibilidad entre distintos navegadores. Además, Bootstrap trabaja de forma nativa bajo el paradigma *Mobile First*.

A diferencia de los métodos anteriores, donde la lógica residía en los archivos `.css`, la maquetación con Bootstrap se realiza directamente en el HTML mediante la asignación de clases compuestas a las etiquetas. Hemos aplicado las siguientes lógicas responsivas:

- **Estructura principal:** Envolvemos todo el contenido en un `<main class=container mt-4>`. La clase `container` centra automáticamente el contenido y aplica márgenes laterales que se adaptan a los diferentes tamaños de pantalla, mientras que `mt-4` añade un margen superior estandarizado.
- **Navegación por pestañas (Tabs):** Utilizamos las clases `nav nav-tabs` para crear el menú visual. Para el interior de los botones, aplicamos utilidades de Flexbox integradas en Bootstrap (`d-flex flex-column align-items-center`) que nos permiten apilar perfectamente el icono del animal sobre su nombre correspondiente.
- **Buscador adaptable (Mobile First):** En la etiqueta `<search>` aplicamos `w-100` para que el buscador ocupe todo el ancho en móviles, y `w-md-50` para que se reduzca a la mitad de la pantalla a partir de monitores medianos.
- **Formulario fluido:** Al formulario interno le asignamos las clases `d-flex flex-column flex-md-row`. Esto es un ejemplo de responsividad mediante clases: por defecto (móviles), la barra de búsqueda y el botón se apilan verticalmente (`flex-column`). Sin embargo, a partir de pantallas medianas (`-md-`), cambian su comportamiento y se colocan uno al lado del otro en horizontal (`flex-md-row`).

A continuación, se presenta el código HTML donde se aprecia la aplicación de estas clases de utilidad:

```
1 <main class="container mt-4">
2 <ul class="nav nav-tabs justify-content-center border-0 gap-3"
3 id="categoria-animales" role="tablist">
4 <li class="nav-item" role="presentation">
5 <button class="nav-link d-flex flex-column align-items-center
6 border rounded p-3 text-muted active"
7 id="home-tab" data-bs-toggle="tab"
8 data-bs-target="#perros-tab-pane" type="button" role="tab"
9 aria-controls="perros-tab-pane" aria-selected="true">
10 
12 Perros
13 </button>
14 </li>
15 <!-- ... otros items omitidos por brevedad ... -->
16 </ul>
17 <search class="w-100 w-md-50 mt-4 m-auto">
```

```

18 <form class="d-flex flex-column flex-md-row bg-white
19 shadow-sm rounded-3 p-1" role="search" name="buscarClinica">
20 <input class="form-control me-2" type="search"
21 placeholder="Buscar consejos..." aria-label="Search" />
22 <button class="btn btn-secondary d-flex justify-content-center
23 p-3 ps-lg-4 pe-lg-4 borde-asimetrico" type="submit">
24 Buscar
25 </button>
26 </form>
27 </search>
28 <ul class="row list-unstyled justify-content-center mt-3 g-3"></ul>
29 </main>

```

Listing 5: Implementación de diseño responsivo mediante clases de Bootstrap

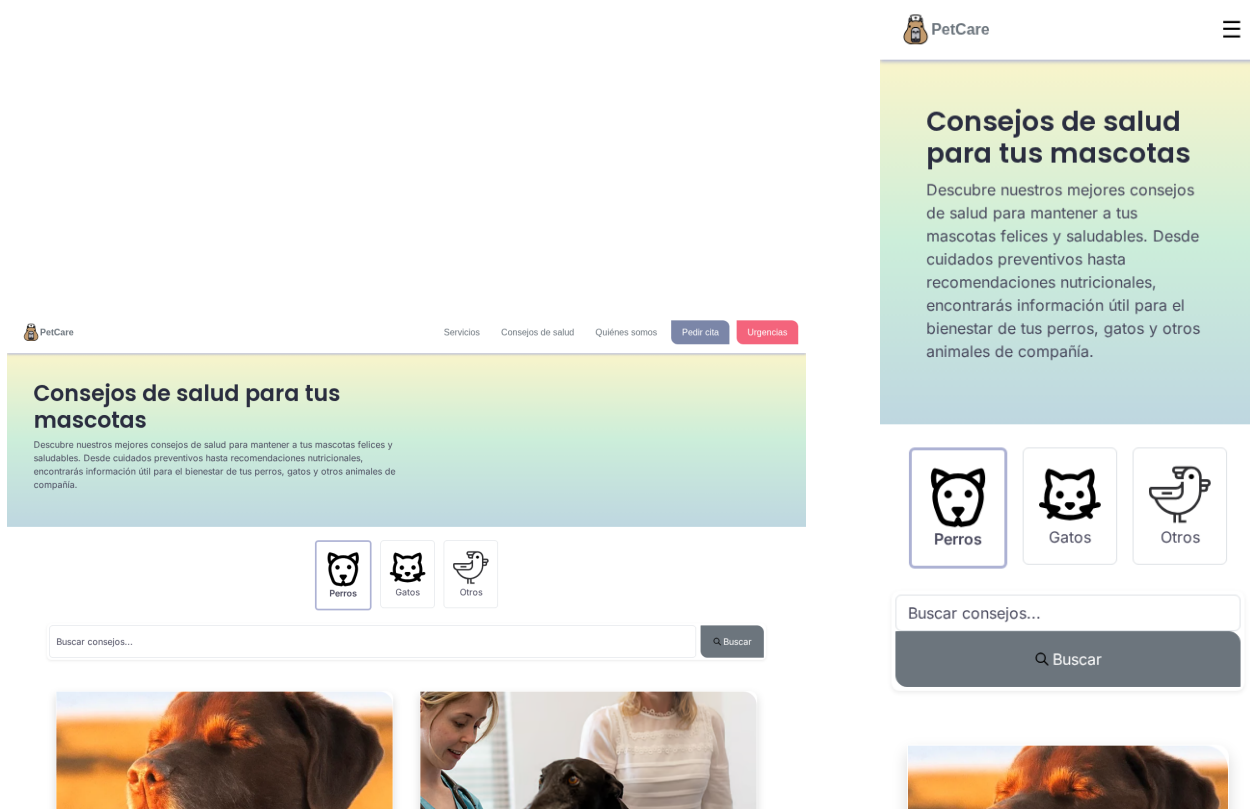


Figura 56: Consejos de salud: versión escritorio (izquierda) y móvil (derecha)

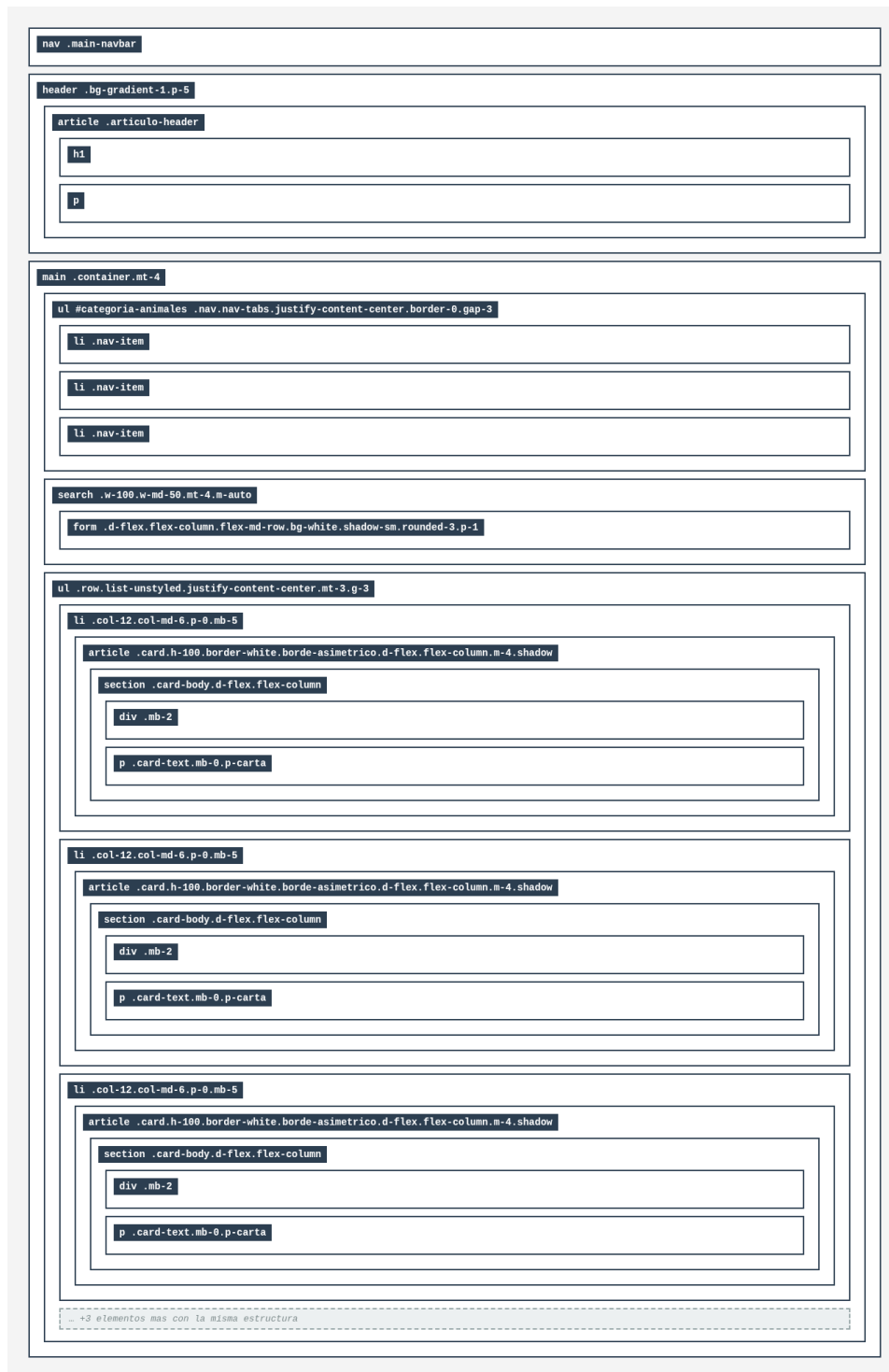


Figura 57: Mapa de etiquetas actualizado (P3) — Consejos de salud

12. Práctica 4: JavaScript

12.1. Efectos visibles

12.1.1. Navegación dinámica por pestañas en la página Sobre Nosotros

En la página “Quiénes somos”, hemos implementado un sistema interactivo de pestañas. Este efecto permite al usuario alternar entre diferentes secciones de texto (Sostenibilidad, Principios, Medicina veterinaria y Por qué PetCare) pulsando sobre sus respectivos botones. Al hacer clic, el contenido inferior se actualiza instantáneamente sin necesidad de recargar la página web, y el botón pulsado se resalta visualmente para indicar en qué sección nos encontramos.

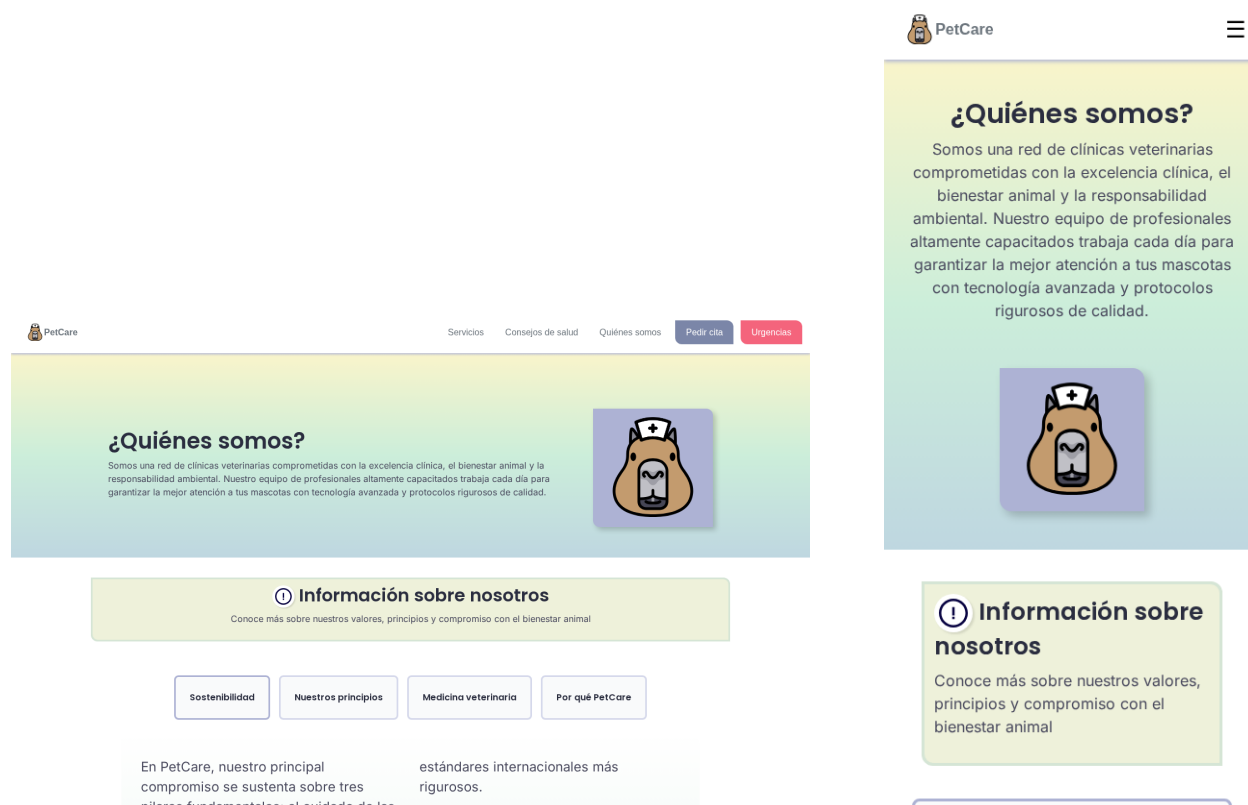


Figura 58: Sobre nosotros: aspecto en escritorio y en móvil con la pestaña activa resaltada

El funcionamiento de este efecto se basa en la interacción coordinada entre la estructura HTML y el motor de eventos de JavaScript. Para lograrlo, dividimos la lógica en las siguientes partes críticas:

- **Preparación del HTML:** En el documento, creamos botones (`<button>`) con la clase común `btn-pestaña`. El elemento clave aquí es el atributo personalizado `data-clave`, que almacena un identificador único para cada botón (por ejemplo, `data-clave="sostenibilidad"`). Además, definimos un contenedor vacío (`<section id=.area-despliegue-texto>`) que actuará como “pizarra” donde inyectaremos el texto.
- **Origen de los datos:** En JavaScript, creamos un objeto constante llamado `repositorioTextos`. Este objeto actúa como un diccionario donde las claves coinciden exactamente con los valores de los atributos `data-clave` del HTML, y sus valores contienen el código HTML del texto a mostrar.

- **Acceso al DOM:** Utilizando el método `querySelectorAll(".btn-pestana")`, capturamos todos los botones en una lista. Simultáneamente, capturamos el contenedor de destino mediante `getElementById("area-despliegue-texto")`.
- **Manejo de Eventos (El clic):** Iteramos sobre la lista de botones usando un bucle `forEach` para asignar a cada uno un escuchador de eventos (`addEventListener`) de tipo `click`.
- **Actualización visual y de contenido:** Cuando se dispara el evento, el código realiza dos acciones. Primero, elimina la clase CSS `activo` de todos los botones y se la añade únicamente al botón que acaba de ser pulsado (`evento.currentTarget`), actualizando así la interfaz. Segundo, lee el atributo `data-clave` de ese botón y utiliza ese valor para buscar el texto correspondiente en el `repositorioTextos`, inyectándolo finalmente en el contenedor mediante la propiedad `innerHTML`.

```

1 <nav class="botones-navegacion">
2 <button class="btn-pestana activo" data-clave="sostenibilidad">
3 Sostenibilidad
4 </button>
5 <button class="btn-pestana" data-clave="principios">
6 Nuestros principios
7 </button>
8 <button class="btn-pestana" data-clave="medicina">
9 Medicina veterinaria
10 </button>
11 <button class="btn-pestana" data-clave="por_que">
12 Por que PetCare
13 </button>
14 </nav>
15 <section id="area-despliegue-texto" class="contenedor-multicol borde-
    asimetrico">
16 </section>

```

Listing 6: Estructura HTML para la botonera y el contenedor de texto

```

1 document.addEventListener("DOMContentLoaded", () => {
2   // 1. Origen de datos (diccionario)
3   const repositorioTextos = {
4     sostenibilidad: '<p>En PetCare, nuestro principal compromiso...</p>',
5     principios: '<p>Como red de clinicas comprometida con...</p>',
6     // ...
7   };
8   // 2. Acceso al DOM
9   const botones = document.querySelectorAll(".btn-pestana");
10  const contenedorDespliegue = document.getElementById("area-despliegue-
    texto");
11
12  // 3. Estado inicial por defecto (Primera pestana activa)
13  botones[0].classList.add("activo");
14  contenedorDespliegue.innerHTML =
15  repositorioTextos[botones[0].getAttribute("data-clave")];
16
17  // 4. Asignacion de manejadores de eventos
18  botones.forEach(boton => {
19    boton.addEventListener("click", (evento) => {

```

```

20      // Actualizacion del estado visual de la botonera
21      botones.forEach(btn => btn.classList.remove("activo"));
22      evento.currentTarget.classList.add("activo");
23      // Recuperacion e inyeccion del contenido asociado
24      const claveSeleccionada =
25      evento.currentTarget.getAttribute("data-clave");
26      contenedorDespliegue.innerHTML = repositorioTextos[
          claveSeleccionada];
27  });
28  });
29  });

```

Listing 7: Lógica JavaScript para la alternancia de pestañas

12.1.2. Búsqueda dinámica de clínicas

En la página de búsqueda de clínicas, hemos desarrollado una interfaz dinámica que permite al usuario encontrar un centro adecuado sin necesidad de recargar la página web. Este sistema se compone de un buscador que filtra los resultados en tiempo real (mientras el usuario teclea), un botón para ordenar alfabéticamente, un filtro para mostrar únicamente las clínicas abiertas en el momento de la consulta, y un botón de reseteo.

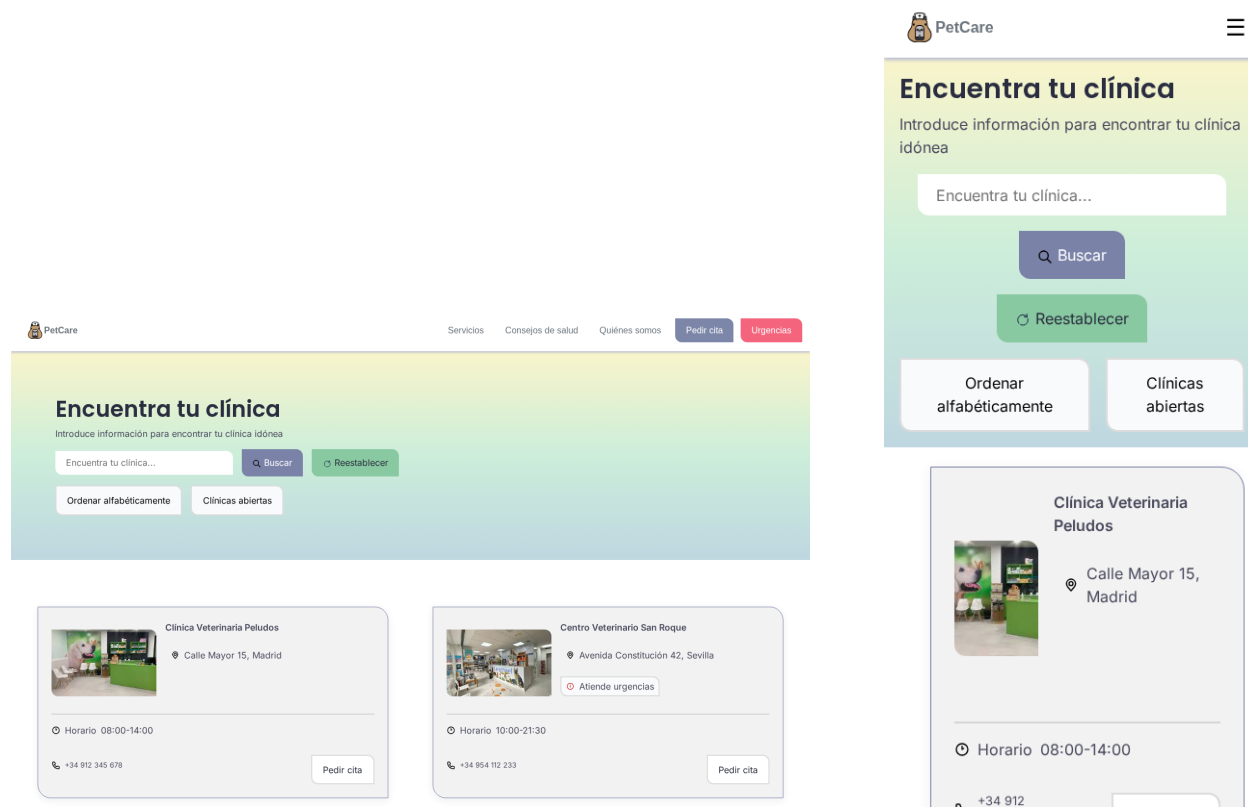


Figura 59: Clínicas: buscador y controles de filtrado en escritorio y en móvil

Este sistema se basa en manipular un array de datos (cargado previamente desde un JSON) y re-renderizar la vista cada vez que el usuario interactúa con los controles. Las partes críticas del código son las siguientes:

- **Buscador en tiempo real:** Utilizando el objeto jQuery `$('#form[name="buscarClinica"]')`, capturamos el formulario de búsqueda. Asignamos un escuchador para el evento `input`, el cual se dispara con cada pulsación de tecla. Extraemos el valor escrito con `$('#.input-busqueda').val()` y utilizamos el método `.filter()` de ES6 sobre el array `clinicasGlobal` para obtener un nuevo array que solo contenga las clínicas cuyo nombre incluya el texto buscado.
- **Ordenación alfabética:** Capturamos el primer botón de filtros mediante `querySelector('.contenedor-filtros:first-child button')`. Al escuchar el evento `click`, utilizamos el método `.sort()` nativo de JavaScript. Dependiendo de una variable booleana (`ordenascendente`), comparamos los nombres de las clínicas pasados a minúsculas y alteramos el orden del array original, re-renderizando las tarjetas inmediatamente después.
- **Filtro de clínicas abiertas:** Seleccionamos el segundo botón de la lista con `querySelectorAll`.

Al recibir un `click`, instanciamos un objeto `new Date()` para obtener la hora actual del sistema del usuario, formateándola en una cadena (ej. “14:30”). Volvemos a usar `.filter()` para separar los horarios de apertura y cierre de cada clínica y comprobar matemáticamente si la hora actual se encuentra dentro de ese rango.

- **Reseteo y renderizado:** El botón “Reestablecer” recarga limpiamente el documento a través de un simple enlace HTML (`href=clnicas.html`). Para redibujar la pantalla tras cada filtro, la función `renderizarTarjetas()` vacía el contenedor utilizando el método `.empty()` de jQuery, clona la etiqueta `<template>` por cada clínica filtrada, e inyecta los nuevos nodos usando `.append()`.

```
1 <search class="contenedor-busqueda">
2 <form name="buscarClinica" class="formulario-busqueda">
3 <input class="boton input-busqueda borde-asimetrico" type="search"
4 placeholder="Encuentra tu clinica..." name="clinica">
5 <button class="boton boton-buscar borde-asimetrico">Buscar</button>
6 </form>
7 <a href="clnicas.html"
8 class="boton boton-reset filtro borde-asimetrico">Reestablecer</a>
9 <menu class="contenedor-filtros">
10 <li><button class="boton borde-asimetrico filtro">
11 Ordenar alfabeticamente</button></li>
12 <li><button class="boton borde-asimetrico filtro">
13 Clinicas abiertas</button></li>
14 </menu>
15 </search>
```

Listing 8: Estructura HTML del buscador y controles de filtrado

```
1 // 1. Buscador en tiempo real (uso de jQuery y evento 'input')
2 function inicializarBuscador() {
3   let formbusqueda = $('form[name="buscarClinica"]');
4   let funcbusqueda = (e) => {
5     e.preventDefault();
6     let txt = $('.input-busqueda').val().toLowerCase().trim();
7     let clinicasFiltradas = clinicasGlobal.filter(clinica =>
8       clinica.nombre.toLowerCase().includes(txt)
9     );
10    renderizarTarjetas(clinicasFiltradas);
11  };
12  formbusqueda.on('input', funcbusqueda);
13 }
14
15 // 2. Filtro de Clinicas abiertas (Manejo de fechas)
16 function inicializarHorario() {
17   let horariosencendido = false;
18   let btn = document.querySelectorAll('.contenedor-filtros li button')
19     [1];
20   btn.addEventListener("click", (e) => {
21     e.preventDefault();
22     horariosencendido = !horariosencendido;
23     if (horariosencendido) {
24       let fechahoy = new Date();
25       let hora = fechahoy.getHours().toString().padStart(2, '0');
```

```

25     let minutos = fechahoy.getMinutes().toString().padStart(2, '0');
26     let horahoytexto = hora + ":" + minutos;
27     let clinicasFiltradas = clinicasGlobal.filter(clinica => {
28         let partes = clinica.horario.split("-");
29         let abre = partes[0].trim();
30         let cierra = partes[1].trim();
31         return horahoytexto >= abre && horahoytexto <= cierra;
32     });
33     renderizarTarjetas(clinicasFiltradas);
34 } else {
35     renderizarTarjetas(clinicasGlobal);
36 }
37 });
38 }

```

Listing 9: Lógica JS para búsqueda en tiempo real y filtrado de horarios

12.1.3. Rotación dinámica de consejos aleatorios en la página de inicio

En la página principal, hemos implementado una sección de “Consejos destacados” que se actualiza de forma automática y autónoma. El sistema selecciona dos consejos de salud al azar y los muestra en pantalla. Cada 10 segundos, estos consejos rotan, dando paso a dos nuevos consejos mediante una transición de fundido.

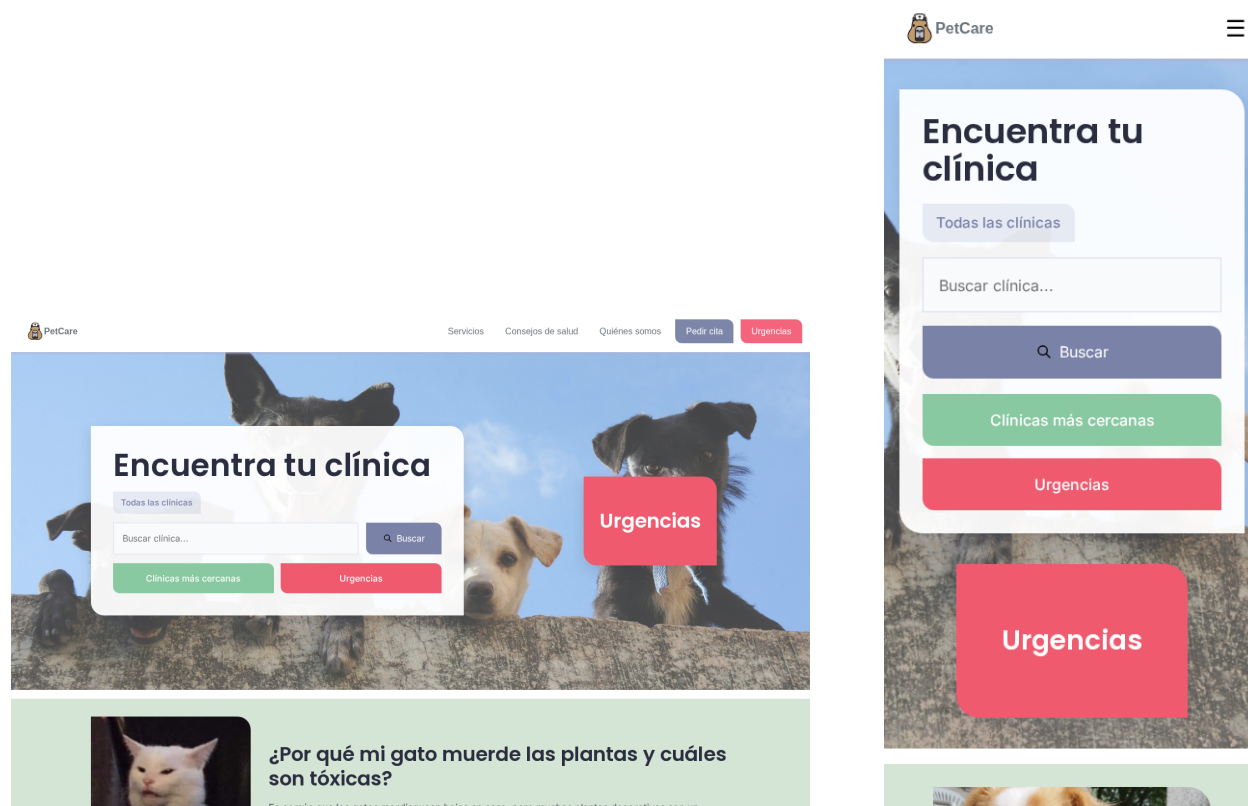


Figura 60: Página principal: sección de consejos destacados rotatorios en escritorio y móvil

Este efecto combina el uso de temporizadores nativos de JavaScript, lógica matemática para la aleatoriedad y manipulación asíncrona del DOM. Las partes críticas del código son:

- **Inicialización y Temporizador:** Una vez cargados los datos desde el servidor, se invoca la función `actualizarConsejosIndex()` para la primera renderización. Inmediatamente después, utilizamos el método `setInterval(actualizarConsejosIndex, 10000)`, el cual programa la ejecución repetitiva de esta función cada 10.000 milisegundos (10 segundos).
- **Lógica de Aleatoriedad:** Para garantizar que los consejos mostrados varían, generamos índices aleatorios utilizando `Math.random()` combinado con `Math.floor()` para redondear a números enteros. Implementamos un bucle `do...while` que asegura que el segundo índice (`indice2`) nunca sea exactamente igual al primero (`indice1`), evitando así mostrar el mismo consejo duplicado en pantalla.
- **Transiciones y Manipulación del DOM:** En la función `renderizarConsejos()`, capturamos el contenedor principal mediante `getElementById(contenedor-consejos-index")`. Para crear el efecto visual de transición, primero añadimos una clase CSS `fade-out` al contenedor. Utilizamos `setTimeout()` para pausar la ejecución de JavaScript durante 300ms (el

tiempo que tarda la animación CSS en ocultar el bloque).

- **Inyección con Template Literals (ES6):** Una vez oculto el contenedor, lo vaciamos (`innerHTML = ''`) y recorremos los dos nuevos consejos con un `forEach`. Utilizamos la sintaxis de plantillas de cadena (*Template Literals*) introducida en ES6 (usando comillas invertidas) para estructurar el HTML e inyectar directamente las variables (como `${consejo.titulo}`) de forma limpia.

```
1 <main>
2 <section class="consejos-home" aria-labelledby="consejos-home-titulo">
3 <h2 id="consejos-home-titulo" class="visually-hidden">
4 Consejos destacados
5 </h2>
6 <div id="contenedor-consejos-index"></div>
7 </section>
8 </main>
```

Listing 10: Contenedor HTML preparado para la inyección dinámica

```
1 // 1. Temporizador ciclico (setInterval)
2 document.addEventListener("DOMContentLoaded", () => {
3     actualizarConsejosIndex();
4     intervaloConsejos = setInterval(actualizarConsejosIndex, 10000);
5 });
6
7 // 2. Logica matematica de aleatoriedad
8 function actualizarConsejosIndex() {
9     if (consejosGlobal.length < 2) return;
10    let indice1 = Math.floor(Math.random() * consejosGlobal.length);
11    let indice2;
12    do {
13        indice2 = Math.floor(Math.random() * consejosGlobal.length);
14    } while (indice1 === indice2);
15    renderizarConsejos([consejosGlobal[indice1], consejosGlobal[indice2]])
16    ;
17 }
18
19 // 3. Transiciones y ES6 Template Literals
20 function renderizarConsejos(listaConsejos) {
21     let contenedor = document.getElementById("contenedor-consejos-index");
22     contenedor.classList.add("fade-out");
23     setTimeout(() => {
24         contenedor.innerHTML = "";
25         listaConsejos.forEach((consejo, indice) => {
26             let dir = indice === 0
27             ? "consejo-home--imagen-izquierda"
28             : "consejo-home--imagen-derecha";
29             contenedor.innerHTML += `
30 <a href="consejo-especifico.html?id=${consejo.id}"
31 class="consejo-home-enlace">
32 <article class="consejo-home ${dir}">
33 <figure class="consejo-home__imagen">
34 
36 </figure>
```

```

36     <section class="consejo-home__texto">
37     <h3>${consejo.titulo}</h3>
38     <p>${consejo.preview}</p>
39     </section>
40     </article>
41     </a>';
42 });
43 contenedor.classList.remove("fade-out");
44 contenedor.classList.add("fade-in");
45 setTimeout(() => contenedor.classList.remove("fade-in"), 500);
46 }, 300);
47 }

```

Listing 11: Lógica JS para la generación aleatoria y transiciones animadas

12.1.4. Autocompletado dinámico del nombre de la clínica en pedir cita

Hemos implementado un sistema de persistencia de datos a corto plazo que conecta la página de Clínicas con la ventana modal del formulario de citas. Cuando el usuario hace clic en el botón “Pedir cita” de una tarjeta específica, el formulario modal se abre mostrando automáticamente el nombre exacto de la clínica seleccionada y su ubicación en la cabecera, evitando que el usuario tenga que introducir o recordar esta información.

Nombre de la clínica

Ubicación de la clínica

Nombre completo

Número de teléfono

Correo electrónico

Nombre de la mascota

Tipo de mascota

Fecha de la cita: dd/mm/yyyy

Hora de la cita: --:--

Pedir cita

Figura 61: Formulario de cita: nombre y ubicación de la clínica autocompletados en escritorio y móvil

Para comunicar la página principal con el *iframe*, hemos utilizado la API `SessionStorage` de HTML5. Las partes críticas del código son:

- **Almacenamiento en origen (`clinicas.js`):** Durante la renderización dinámica de las tarjetas, localizamos el botón de cita mediante `querySelector(".pedir-cita")` y le asignamos un evento `click`. Al dispararse el evento, en lugar de modificar el DOM directamente, utilizamos `sessionStorage.setItem()` para guardar el nombre y la ubicación de esa clínica específica en la memoria temporal del navegador.
- **Recuperación en destino (`pedir-cita.js`):** El documento `pedir-cita.html` (que vive dentro del *iframe*) cuenta con su propio archivo JavaScript. Escuchamos el evento `DOMContentLoaded` para asegurarnos de que el formulario base ya se ha dibujado.
- **Lectura e Inyección en el DOM:** Utilizamos `sessionStorage.getItem()` para extraer las variables almacenadas previamente. Mediante una comprobación lógica (`if (nombreclinica)`), validamos que los datos existan. De ser así, localizamos los elementos del DOM correspon-

dientes usando `querySelector(".titulo")` y `getElementById('texto-ubicacion-cita')` y actualizamos su texto en pantalla mediante la propiedad `textContent`.

```
1 // Fragmento dentro del bucle de renderizarTarjetas()
2 let btncita = copia.querySelector(".pedir-cita");
3 btncita.addEventListener("click", () => {
4     sessionStorage.setItem('nombreclinica', clinica.nombre);
5     sessionStorage.setItem('ubicacionclinica', clinica.ubicacion);
6 });
```

Listing 12: Guardado de estado en la generación de la tarjeta (clinicas.js)

```
1 document.addEventListener("DOMContentLoaded", function() {
2     let nombreclinica = sessionStorage.getItem('nombreclinica');
3     let ubicacionclinica = sessionStorage.getItem('ubicacionclinica');
4
5     if (nombreclinica) {
6         document.querySelector(".titulo").textContent = nombreclinica;
7     }
8     if (ubicacionclinica) {
9         document.getElementById('texto-ubicacion-cita').textContent =
10         ubicacionclinica;
11     }
12 });
```

Listing 13: Recuperación e inyección de datos en el iframe modal (pedir-cita.js)

12.2. Métodos de acceso al DOM

Para manipular los elementos de la página, hemos utilizado los 5 métodos principales que ofrece la API nativa del DOM, seleccionando en cada caso el más eficiente según la necesidad:

- `getElementById()`: Utilizado en `sobre-nosotros.js` para capturar el contenedor único donde se inyecta el texto (`document.getElementById('area-despliegue-texto')`). Es el método más rápido y directo al apuntar a un ID único.
- `querySelector()`: Empleado en `main.js` para seleccionar el botón del menú móvil a través de su clase (`document.querySelector('.menu-hamburguesa-moviles')`). Devuelve la primera coincidencia que encuentra.
- `querySelectorAll()`: Usado en `sobre-nosotros.js` para recopilar todos los botones de las pestañas (`document.querySelectorAll('.btn-pestana')`). Nos devuelve un `NodeList` que podemos iterar fácilmente.
- `getElementsByTagName()`: Utilizado en `clinicas.js` para localizar la etiqueta oculta de renderizado (`document.getElementsByTagName("template")[0]`).
- `getElementsByClassName()`: Empleado en la función de limpieza de `clinicas.js` para acceder a las barras de búsqueda (`document.getElementsByClassName('input-busqueda')`).

12.3. Eventos implementados

Para lograr que la página sea interactiva, el código se mantiene “escuchando” las acciones del usuario y del propio navegador. Se han implementado múltiples eventos, destacando los tres siguientes por su papel fundamental en la arquitectura de la web:

1. `DOMContentLoaded`: Presente en la cabecera de todos los archivos *script* (ej. `main.js`, `clinicas.js`). Garantiza que nuestro código JavaScript solo comience a ejecutarse cuando el navegador haya terminado de descargar y construir el árbol DOM, evitando errores por intentar acceder a elementos que aún no existen.
2. `click`: Utilizado para detectar interacciones directas del usuario con botones o enlaces. Por ejemplo, en `main.js` para desplegar el menú hamburguesa móvil (`botonHamburguesa.addEventListener('click', ...)`), o en `clinicas.js` para los botones de filtrado y ordenación.
3. `input`: Utilizado en `clinicas.js` y `servicios.js` (`inputBusqueda.addEventListener('input', ...)`). A diferencia de `keyup` o `change`, este evento se dispara inmediatamente con cada modificación del texto en la barra de búsqueda (ya sea tecleando, pegando o borrando), permitiéndonos ofrecer un filtrado reactivo en tiempo real.

12.4. Sintaxis ECMAScript 6

Destacan las siguientes implementaciones:

- **Funciones Flecha (*Arrow Functions*)**: Se han utilizado extensamente en los *callbacks* de eventos y métodos de arrays, eliminando la necesidad de escribir la palabra clave `function` y manteniendo el contexto de `this`.
- **Variables de bloque (`let` y `const`)**: Sustituyendo al tradicional `var` para garantizar que las variables respeten el ámbito de su bloque, previniendo errores de reasignación.
- **Async/Await**: En `servicio-especifico.js` y `consejos.js`, hemos utilizado esta sintaxis moderna para manejar la asincronía de las peticiones `fetch`, logrando que el código asíncrono se lea de forma estructurada como si fuera síncrono.

```

1 // Arrow function aplicada al iterador forEach
2 serviciosFiltrados.forEach(servicio => {
3   const li = document.createElement("li");
4   // Template Literal para la inyeccion dinamica multilineal
5   li.innerHTML = `
6     <a class="tarjeta-servicio borde-asimetrico-2"
7     href="servicio-especifico.html?id=${servicio.id}">
8     
10    <h3 class="titulo-tarjeta">${servicio.nombre}</h3>
11    <p>${servicio.descripcion}</p>
12    </a>
13  `;
14   lista.appendChild(li);
15 });

```

Listing 14: Ejemplo del uso de ES6 con Arrow Functions y Template Literals en servicios.js

12.5. Uso de objetos jQuery

Aunque gran parte del proyecto se ha desarrollado en JS nativo hemos incorporado la librería **jQuery** en puntos estratégicos del código, específicamente en las vistas de clínicas y urgencias.

- **Selección de nodos y Colecciones:** Hemos utilizado el constructor global `$(...)` para capturar elementos de forma concisa. Por ejemplo, en `clinicas.js`, asignamos a variables objetos de jQuery como `let contenedor = $(".contenedor-tarjetas");` o `let formbusqueda = $('form[name="buscarClinica"]');`
- **Métodos de manipulación estructural:** Durante el re-renderizado de tarjetas, jQuery nos ahorra escribir bucles de eliminación de nodos hijos gracias a su método `.empty()`. Una vez generadas las nuevas tarjetas clónicas nativas, las inyectamos fácilmente usando `contenedor.append(copia)`.
- **Lectura rápida de valores:** En el buscador, extraer y sanitizar el texto del input se reduce a una sola línea gracias al método `.val()` concatenado con funciones de cadena nativas: `$('#input-busqueda').val().toLowerCase().trim()`.
- **Asignación de eventos:** En lugar del verboso `addEventListener`, el objeto jQuery de nuestro formulario delega los eventos de teclado de forma compacta mediante el método `.on()`: `formbusqueda.on('input', funcbusqueda);`

```

1 function inicializarBuscador() {
2   // 1. Creacion de un objeto jQuery desde un selector CSS
3   let formbusqueda = $('form[name="buscarClinica"]');
4
5   let funcbusqueda = (e) => {
6     e.preventDefault();
7     // 2. Uso del metodo .val() propio de jQuery
8     let txt = $('#input-busqueda').val().toLowerCase().trim();
9     let clinicasFiltradas = clinicasGlobal.filter(clinica =>
10      clinica.nombre.toLowerCase().includes(txt)
11    );
12    renderizarTarjetas(clinicasFiltradas);
13  };

```

```
14
15  // 3. Asignacion de eventos con el metodo .on()
16  formbusqueda.on('submit', funcbusqueda);
17  formbusqueda.on('input', funcbusqueda);
18 }
```

Listing 15: Manipulación del DOM y eventos mediante jQuery en clinicas.js

12.6. Carga de contenido en formato XML

Para la sección de “Servicios”, la información detallada de cada especialidad veterinaria se ha estructurado y almacenado en un archivo externo en formato XML (`servicios.xml`).

A continuación, se muestra un extracto simplificado de la estructura del archivo XML, donde cada especialidad está encapsulada en una etiqueta `<servicio>` provista de un atributo `id` único, conteniendo a su vez etiquetas descriptivas y múltiples bloques de texto:

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <servicios>
3 <servicio id="neurologia">
4 <nombre>Neurologia</nombre>
5 <imagen>assets/imgs/servicios-neurologia.png</imagen>
6 <descripcion-corta>Evaluacion y tratamiento de trastornos...</
  descripcion-corta>
7 <subtitulo>Expertos en el cuidado del sistema nervioso</subtitulo>
8 <bloque>
9 <titulo>1. Evaluacion Neurologica Completa</titulo>
10 <texto>Realizamos exámenes exhaustivos de los reflejos...</texto>
11 </bloque>
12 </servicio>
13 </servicios>
```

Listing 16: Estructura del archivo de datos servicios.xml

12.6.1. Método utilizado: Objeto XMLHttpRequest

Para cumplir con la exigencia de implementar diversas formas de carga asíncrona de datos, en la página general de Servicios hemos consumido este archivo XML utilizando el objeto nativo `XMLHttpRequest`.

La implementación sigue un flujo basado en eventos y comprobaciones de estado:

- **Instanciación y Configuración:** Creamos una nueva instancia mediante `new XMLHttpRequest()` y configuramos la petición con el método `.open("GET", "assets/xml/servicios.xml", true)`, indicando que es una petición asíncrona (`true`).
- **Escucha de cambios de estado:** Asignamos una función al controlador de eventos `.onreadystatechange`. El navegador invocará esta función en cada fase del ciclo de vida de la petición.
- **Validación:** Dentro del evento, comprobamos de forma estricta que `xhr.readyState === 4` (operación de red completada) y que `xhr.status === 200` (código de éxito HTTP).
- **Parseo automático:** Una de las grandes ventajas de `XMLHttpRequest` al trabajar con XML es su propiedad `.responseXML`. Si el documento XML está bien formado, el navegador lo convierte automáticamente en un árbol de nodos manipulable.

```
1 function cargarServicios() {
2   let xhr = new XMLHttpRequest();
3   xhr.open("GET", "assets/xml/servicios.xml", true);
4   xhr.onreadystatechange = function () {
5     if (xhr.readyState === 4 && xhr.status === 200) {
6       let documentoXML = xhr.responseXML;
7       procesarServicios(documentoXML);
8     }
9   }
10 }
```



```

9   };
10  xhr.onerror = function() {
11      console.error("Error al cargar servicios:", xhr.statusText);
12  };
13  xhr.send();
14  }
15
16  function procesarServicios(xmlDoc) {
17      let listaServicios = xmlDoc.getElementsByTagName("servicio");
18      Array.from(listaServicios).forEach(servicio => {
19          let id      = servicio.getAttribute("id");
20          let nombre = servicio.getElementsByTagName("nombre")[0]
21              .textContent.trim();
22          // ... almacenamiento y renderizado omitidos
23      });
24  }

```

Listing 17: Carga asíncrona y parseo de XML mediante XMLHttpRequest en servicios.js

Nota metodológica: Para la página de detalle de un servicio individual (`servicio-especifico.js`), hemos utilizado una tercera aproximación técnica: la moderna **API Fetch** combinada con la clase **DOMParser** para transformar la respuesta de texto en un documento XML.

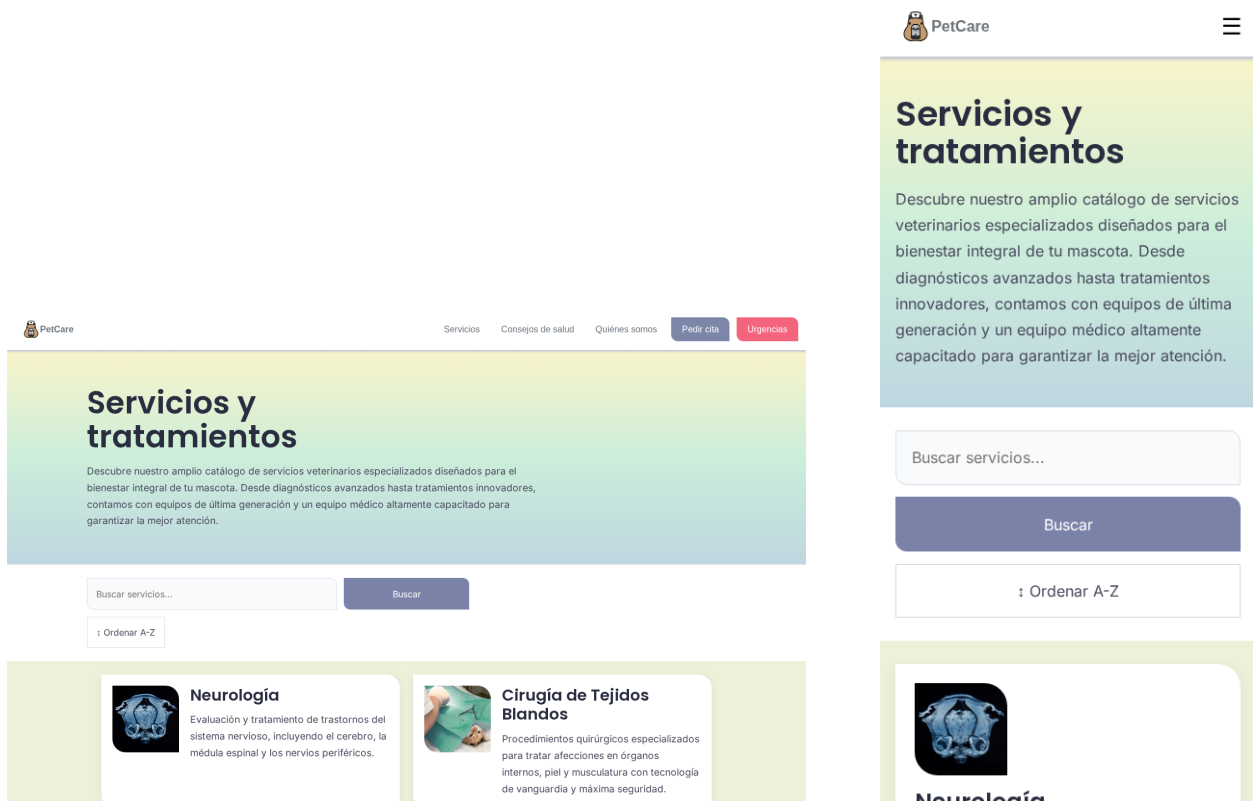


Figura 62: Servicios: carga dinámica desde XML en escritorio y en móvil

12.7. Carga de contenido en formato JSON

Para dotar al sitio web de información dinámica y fácilmente actualizable, los datos estructurados de los centros veterinarios se han externalizado en un archivo local llamado `clinicas.json`. Este archivo actúa como una base de datos de solo lectura que alimenta tanto a la página de “Clínicas” como a la de “Urgencias”.

A continuación, se muestra un fragmento representativo de la estructura de este archivo JSON:

```
1 {
2   "clinicas": [
3     {
4       "nombre": "Clinica Veterinaria Peludos",
5       "ubicacion": "Calle Mayor 15, Madrid",
6       "foto": "assets/imgs/clinica-1.jpeg",
7       "horario": "08:00-14:00",
8       "telefono": "+34 912 345 678",
9       "urgencias": false
10    },
11    {
12      "nombre": "Centro Veterinario San Roque",
13      "ubicacion": "Avenida Constitucion 42, Sevilla",
14      "foto": "assets/imgs/clinica-2.jpeg",
15      "horario": "10:00-21:30",
16      "telefono": "+34 954 112 233",
17      "urgencias": true
18    }
19  ]
20 }
```

Listing 18: Estructura simplificada del archivo `clinicas.json`

12.7.1. Método utilizado: objeto jQuery (`$.getJSON`)

El enunciado de la práctica requiere utilizar distintas formas para la carga de datos. Para el caso específico de las clínicas y urgencias, hemos optado por utilizar la librería **jQuery**, concretamente su método asíncrono `$.getJSON()`.

Este método realiza la petición HTTP y, de forma automática, parsea la cadena de texto JSON transformándola en un objeto nativo de JavaScript listo para ser iterado, ahorrándonos el paso de usar `JSON.parse()`.

```
1 let clinicasGlobal = [];
2 document.addEventListener("DOMContentLoaded", function() {
3   $.getJSON("assets/json/clinicas.json", (datos) => {
4     clinicasGlobal = datos.clinicas;
5     renderizarTarjetas(clinicasGlobal);
6     inicializarBuscador();
7     inicializarOrden();
8     inicializarHorario();
9   });
10 });
```

Listing 19: Carga asíncrona de JSON mediante jQuery en `clinicas.js`

Implementación en la página de Urgencias

En el archivo `urgencias.js`, reutilizamos el mismo archivo `clinicas.json` mediante `$.getJSON()`, pero aplicamos una lógica de negocio diferente en el *callback*. En lugar de mostrar todas las clínicas, iteramos sobre los datos recibidos utilizando el método `.filter()` para extraer únicamente aquellas que cumplan dos condiciones simultáneas:

1. Que su atributo `urgencias` sea `true`.
2. Que la hora actual del sistema del usuario esté dentro del rango definido en su atributo `horario`.

```
1 document.addEventListener("DOMContentLoaded", function () {
2   $.getJSON("assets/json/clinicas.json", (datos) => {
3     const clinicas = datos.clinicas;
4     const ahora = new Date();
5     const minutosActuales = (ahora.getHours() * 60) + ahora.getMinutes()
6       ;
7     const clinicasDisponibles = clinicas.filter((clinica) => {
8       return clinica.urgencias &&
9         estaDisponibleAhora(clinica.horario, minutosActuales);
10    });
11
12    clinicasDisponibles.forEach((clinica) => {
13      // (Logica de clonacion del template omitida)
14    });
15  });
16 });
```

Listing 20: Filtrado en tiempo de ejecución tras la carga del JSON en `urgencias.js`

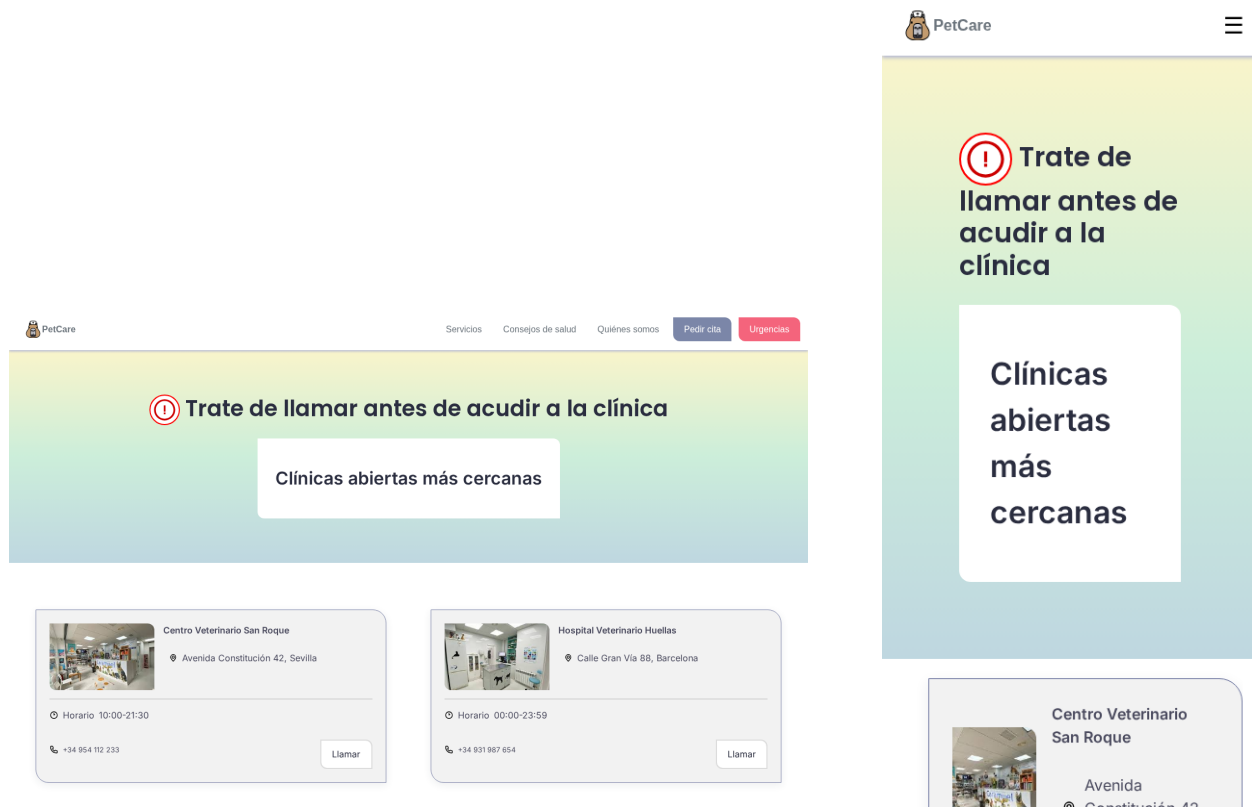


Figura 63: Urgencias: clínicas filtradas dinámicamente desde JSON en escritorio y móvil